

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
PHÂN HIỆU TẠI TP. HỒ CHÍ MINH
BỘ MÔN CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN TỐT NGHIỆP

ĐỀ TÀI: Phát triển game chiến thuật Turn-Based RPG, áp dụng AI Utility Scoring ,quản lý dữ liệu bằng Custom Tool trong Unity.

Giảng viên hướng dẫn: Th.S TRẦN PHONG NHÃ

Sinh viên thực hiện: PHAN VỸ KIỆT

Lớp : CQ.CNTT.62

Khoá : K62

Tp. Hồ Chí Minh, năm 2026

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
PHÂN HIỆU TẠI TP. HỒ CHÍ MINH
BỘ MÔN CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN TỐT NGHIỆP

**ĐỀ TÀI: Phát triển game chiến thuật Turn-Based RPG, áp dụng AI
Utility Scoring, quản lý dữ liệu bằng Custom Tool trong Unity.**

Giảng viên hướng dẫn: TRẦN PHONG NHÃ

Sinh viên thực hiện: PHAN VỸ KIỆT

Lớp : CQ.CNTT.62

Khoá : K62

Tp. Hồ Chí Minh, năm 2026

NHIỆM VỤ THIẾT KẾ TỐT NGHIỆP

BỘ MÔN: CÔNG NGHỆ THÔNG TIN

-----***-----

Mã sinh viên:6251071050

Họ tên SV: Phan Võ Kiệt

Khóa: K62

Lớp: CQ.CNTT.62

1. Tên đề tài :

Phát triển game chiến thuật Turn-Based RPG, áp dụng AI Utility Scoring, quản lý dữ liệu bằng Custom Tool trong Unity

2. Mục đích, yêu cầu:

Xây dựng một game chiến thuật theo lượt có thể chơi được , áp dụng các kiến thức lập trình, hướng đối tượng, cấu trúc dữ liệu và thuật toán. Nghiên cứu và triển khai hệ thống AI enemy có khả năng ra quyết định một cách thông minh. Cố gắng thiết kế hệ thống custom tool và edit dễ hiểu , rõ ràng , dễ mở rộng cuối cùng là áp dụng bối cảnh Việt Nam vào trò chơi.

3. Nội dung và phạm vi đề tài:

Xây dựng Battle Scene theo cơ chế Turn-Base , cùng với đó là xây dựng Story Scene theo cơ chế rpg cho em va chạm và tương tác , thiết kế hệ thống Encounter cho người chơi va chạm để chuyển sang Battle scene , xây dựng hệ thống Enemy AI dựa trên Utility Scoring System và Threat System , phạm vi đề tài chủ yếu vào gameplay và phản ứng của AI ,chưa đi sâu vào đồ họa nâng cao.

4. Công nghệ, công cụ và ngôn ngữ lập trình:

Ngôn ngữ lập trình: C#

Engine:Unity

Database: SQLite

Công cụ hỗ trợ: Unity Editor, Visual Studio

Mô hình AI: Utility AI, Threat-based AI

5. Các kết quả chính dự kiến sẽ đạt được và ứng dụng

Game ở trạng thái ổn định và có thể chơi được , hệ thống theo lượt mượt mà dựa trên biến tốc độ , AI của enemy có khả năng ra quyết định linh hoạt dựa trên hành động của người chơi story Scene kết nối liền mạch với Battle Scene, tạo nền tảng để phát triển thêm nội dung trong tương lai

6. Giáo viên và cán bộ hướng dẫn

Họ tên: Trần Phong Nhã

Đơn vị công tác: Bộ môn Công nghệ thông tin

Điện thoại: 0906.761.014

Email: tpnha@utc2.edu.vn

Ngày tháng 06 năm 2025

Trưởng BM Công nghệ Thông tin

Đã giao nhiệm vụ TKTN

Giáo viên hướng dẫn

ThS. Trần Phong Nhã

ThS. Trần Phong Nhã

Đã nhận nhiệm vụ TKTN

Sinh viên: Phan Vũ Kiệt

Điện thoại: 0937824006

Ký tên:

Email: phankiet24006@gmail.com

LỜI CẢM ƠN

Trong suốt quá trình học tập và thực hiện đồ án tốt nghiệp, em đã nhận được rất nhiều sự quan tâm, hướng dẫn và giúp đỡ quý báu từ thầy cô, gia đình và bạn bè. Đây là nguồn động viên to lớn giúp em hoàn thành đề tài của mình.

Trước hết, em xin bày tỏ lòng biết ơn sâu sắc đến **ThS. Trần Phong Nhã**, giảng viên hướng dẫn, người đã tận tình chỉ bảo, định hướng và đồng hành cùng em trong suốt quá trình thực hiện đồ án. Thầy không chỉ dành nhiều thời gian hướng dẫn, góp ý và chỉnh sửa nội dung mà còn đưa ra nhiều ý tưởng, định hướng phát triển đề tài cũng như chia sẻ những kinh nghiệm thực tiễn quý báu. Những nhận xét và góp ý của thầy đã giúp em từng bước hoàn thiện đồ án, nâng cao kiến thức chuyên môn cũng như rèn luyện tư duy nghiên cứu và giải quyết vấn đề.

Em xin chân thành cảm ơn quý thầy cô trong khoa đã tận tâm giảng dạy, truyền đạt cho em những kiến thức nền tảng và chuyên môn trong suốt thời gian học tập tại trường. Những kiến thức và kỹ năng mà thầy cô đã trang bị là hành trang quý giá, giúp em có đủ nền tảng để thực hiện và hoàn thành đề tài tốt nghiệp.

Bên cạnh đó, em xin gửi lời cảm ơn sâu sắc đến gia đình vì luôn là chỗ dựa vững chắc, không ngừng động viên, quan tâm và tạo mọi điều kiện thuận lợi để em yên tâm học tập và nghiên cứu. Em cũng xin cảm ơn bạn bè và các anh chị đã luôn sẵn sàng chia sẻ kinh nghiệm, hỗ trợ, đóng góp ý kiến và động viên em trong suốt quá trình thực hiện đồ án.

Mặc dù đã nỗ lực hoàn thành đồ án với tinh thần học hỏi và trách nhiệm cao nhất, song do kiến thức và kinh nghiệm thực tế còn hạn chế nên báo cáo khó tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp và nhận xét quý báu từ quý thầy cô để đề tài được hoàn thiện hơn.

Cuối cùng, em xin kính chúc quý thầy cô luôn dồi dào sức khỏe, hạnh phúc và gặt hái nhiều thành công trong sự nghiệp giảng dạy. Em xin chân thành cảm ơn.

[illegible]

Giáo viên hướng dẫn

4

MỤC LỤC

LỜI CẢM ƠN	3
MỤC LỤC	5
DANH MỤC CHỮ VIẾT TẮT.....	7
BẢNG BIỂU, SƠ ĐỒ, HÌNH VẼ	8
TỔNG QUAN ĐỀ TÀI.....	9
CHƯƠNG 1. CƠ SỞ LÝ THUYẾT	11
1.1 Tổng quan về game chiến thuật theo lượt (Turn based strategy) :	11
1.2 Trí Tuệ nhân Tạo trong Game :	13
1.3 Mô hình Utility AI :	14
1.4 Thiết kế kiến trúc hệ thống game :	17
1.5 Công nghệ và công cụ sử dụng :	19
CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	22
2.1 Mục tiêu và vai trò của giai đoạn phân tích thiết kế :	22
2.2. Phân tích yêu cầu hệ thống.....	22
2.3. Phân tích luồng hoạt động của hệ thống.....	24
2.4. Thiết kế kiến trúc hệ thống chi tiết.....	29
2.5. Thiết kế hệ thống chiến đấu theo lượt.....	30
2.6. Thiết kế hệ thống trí tuệ nhân tạo theo mô hình Utility AI	32
2.7. Thiết kế dữ liệu và lớp đối tượng	36
2.8. Tổng kết chương	38
CHƯƠNG 3: XÂY DỰNG VÀ CÀI ĐẶT HỆ THỐNG	39
3.1 Kiến trúc tổng thể của hệ thống :	39
3.2 Xây dựng hệ thống Battle :	39
3.3. Xây dựng hệ thống nhân vật :	41
3.4 Xây dựng hệ thống kỹ năng :	43
3.5. Xây dựng hệ thống AI Enemy.....	46
3.6 Hình ảnh Gameplay của dự án :	50
CHƯƠNG 4: KẾT LUẬN VÀ KHUYẾN NGHỊ.....	58
4.1. Kết quả đạt được	58

4.2. Đánh giá mức độ hoàn thành mục tiêu.....	58
4.3. Hạn chế của đề tài	59
4.4. Hướng phát triển trong tương lai.....	59
TÀI LIỆU THAM KHẢO.....	65

DANH MỤC CHỮ VIẾT TẮT

STT	Mô tả	Ý nghĩa	Ghi chú
1	AI	Artificial Intelligence (Trí tuệ nhân tạo)	
2	RPG	Role-Playing Game (Trò chơi nhập vai)	
3	CRUD	Create, Read, Update, Delete	
4	UI	User Interface (Giao diện người dùng)	
5	NPC	Non-Player Character (Nhân vật không do người chơi điều khiển)	
6	HP	Health Points (Điểm sinh lực)	
7	FSM	Finite State Machine (Máy trạng thái hữu hạn)	
8	ERD	Entity Relationship Diagram (Sơ đồ thực thể – liên kết)	
9	USS	Utility Scoring System (Hệ thống tính điểm Utility)	
10	AoE	Area of Effect (Phạm vi tác động)	
11	SQL	Structured Query Language (Ngôn ngữ truy vấn có cấu trúc)	
12	DB	Database (Cơ sở dữ liệu)	

BẢNG BIỂU, SƠ ĐỒ, HÌNH VẼ

Hình 1:Luồng dữ liệu biến utility	16
Hình 2 : bảng so sánh các mô hình.....	17
Hình 3:Sơ đồ luồng xử lý dữ liệu	18
Hình 4 :Sơ đồ phân rã chức năng	23
Hình 5 : Sơ đồ Luồng dữ liệu	25
Hình 6 : Sơ đồ tuần tự.....	26
Hình 7 : Hình mạch xử lý tác vụ.....	29
Hình 8 : Sơ đồ ERD.....	36
Hình 9 :Class Diagram	37
Hình 10 :Cơ chế hàng đợi trong turn based.....	40
Hình 11 : sơ đồ luồng xử lý của AI	48
Hình 12: Main Menu	50
Hình 13 Starter Panel.....	50
Hình 14 Intro Panel cốt truyện	51
Hình 15 Main scene - Story Scene	51
Hình 16 :Team building panel.....	52
Hình 17 Character Panel.....	52
Hình 18 Quest panel	53
Hình 19 Save/Load panel	53
Hình 20 : Inventory	53
Hình 21 Shop Panel	54
Hình 22 Main Battle Scene.....	54
Hình 23 Skill Panel.....	55
Hình 24 Skill Editor.....	55
Hình 25 NPC Editor.....	56
Hình 26 Quest Editor	56
Hình 27 Dialog Editor	57

TỔNG QUAN ĐỀ TÀI

Trong những năm gần đây, game chiến thuật theo lượt (Turn-Based Strategy) ngày càng được ưa chuộng nhờ chiều sâu chiến thuật xây dựng nhân vật và phát triển cốt truyện. Không giống các game thẻ loại thời gian thực thì chiến thuật theo lượt yêu cầu người chơi phải có các tính toán và các ý tưởng quyết định hợp lý trong từng lượt thực thi nhờ thế nên tạo được chiều sâu chiến thuật của thẻ loại .

Hệ thống chiến đấu không phải chỉ là thứ quan trọng nhất mà còn có trí tuệ nhân tạo (Artificial Intelligence – AI) có vai trò quan trọng trong việc tạo ra các đối thủ sừng sỏ có khả năng linh hoạt trước các cách hành động của người chơi . Các thẻ hệ thống AI truyền thống vẫn dựa trên mô hình Rule based AI dẫn đến các hành vi lặp lại dễ đoán và khó để nâng cấp và mở rộng khi đề tài cập nhật thêm về kỹ năng, các cơ chế chiến đấu đặc biệt , đó là lý do một mô hình khác Utility Scoring System được nghiên cứu và ứng dụng , nhằm tăng cao tính chiến thuật , giảm khả năng bắt bài của người chơi ,tạo cho kẻ thù khả năng thích nghi với người chơi .

Ngoài ra hành trình phát triển game có một thứ luôn được đầu tư là dữ liệu của hầu hết mọi thứ ở trong game , việc quản lý một khối lượng lớn dữ liệu như nhân vật, kỹ năng thường tiêu tốn khá nhiều thời gian vì phải thực hiện thủ công. Vì thế công cụ hỗ trợ phát triển (Custom Tool) đảm nhiệm các công đoạn CRUD được thực thi để tối ưu quá trình làm việc .

Từ những yếu tố trên , đề tài “Phát triển game nhập vai chiến thuật theo lượt (Turn-Based RPG), có áp dụng mô hình AI Utility Scoring System “ đã được lựa chọn thực hiện nhằm nghiên cứu, thiết kế và xây dựng một hệ thống game có yếu tố chiến thuật, khả năng mở rộng cao và tích hợp các công cụ hỗ trợ phát triển hiệu quả .

Mục tiêu nghiên cứu:

Nghiên cứu cơ chế Turn base trong các game chiến thuật ,kể đến là các hệ thống và thuật toán liên quan đến Utility AI , thiết kế kiến trúc của game rõ ràng để mở rộng về sau này.

Phạm vi:

Đối tượng nghiên cứu là cơ chế gameplay chiến thuật theo lượt và hệ thống trí tuệ nhân tạo được sử dụng trong game chiến thuật .

Đề tài tập trung vào Xây dựng Game play chiến đấu và Hệ thống AI , không tập trung vào đồ họa 2D , 3D nâng cao .Sản phẩm hướng đến một game ở mức độ prototype hoàn chỉnh.

Phương pháp nghiên cứu :

Nghiên cứu cơ chế hoạt động của game chiến thuật theo lượt , tìm hiểu và áp dụng các thuật toán , mô hình liên quan đến Utility AI trong game phân tích và so sánh các hệ thống AI phổ biến trong game cùng thể loại .Thiết kế và cài đặt hệ thống game bằng công cụ Unity và ngôn ngữ C# , kiểm thử và đánh giá ,thay đổi hệ thống qua quá trình vận hành Game .

Cấu trúc báo cáo thực tập tốt nghiệp

1. Tổng Quan Đề Tài: là về giới thiệu tổng quan đề tài, lý do chọn đề tài, mục tiêu, phạm vi nghiên cứu và cấu trúc báo cáo .
2. Chương 1: Trình bày các cơ sở lí thuyết về các công cụ công nghệ, ngôn ngữ, hệ thống về game chiến thuật theo lượt ,cùng với đó là các mô hình trí tuệ nhân tạo trong game .
3. Chương 2: Phân tích yêu cầu, thiết kế kiến trúc hệ thống game và thiết kế hệ thống AI kẻ thù .
4. Chương 3: Trình bày quá trình lên ý tưởng kế đến là xây dựng và cài đặt game trên Unity, triển khai gameplay và hệ thống AI kẻ thù.
5. Chương 4 : Hình ảnh của dự án .
6. Chương 5: Đánh giá kết quả đạt được cùng với đó là so sánh mục tiêu đề ra và đề xuất hướng phát triển trong tương lai.

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT

1.1 Tổng quan về game chiến thuật theo lượt (Turn based strategy) :

Khái niệm game chiến thuật theo lượt :

Game chiến thuật theo lượt là thể loại game trong đó người chơi và các đối thủ thực hiện hành động luân phiên theo từng lượt riêng biệt . Không giống như game thời gian thực thì thể loại này không yêu cầu phản xạ nhanh kết hợp hành động tay và theo Milington và Funge thì các trò chơi chiến thuật theo lượt cho phép người chơi đưa ra quyết định dựa trên phân tích hệ thống thay vì phản xạ thời gian thực từ đó tạo ra chiều sâu cho thể loại chiến thuật. [1]

Đặc trưng nổi bật của Turn based nằm ở việc mọi hành động đều mang tính chiến thuật rõ ràng và có ảnh hưởng lâu dài đến cục diện trận đấu. Mỗi quyết định được đưa ra không chỉ giải quyết tình huống hiện tại mà còn tác động đến các lượt tiếp theo .Điều này tạo nên chiều sâu chiến thuật và yêu cầu người chơi phải có tư duy lập kế hoạch dài hạn [1]

Trong bối cảnh phát triển game hiện đại, game chiến thuật theo lượt vẫn giữ được vị trí quan trọng nhờ khả năng mở rộng gameplay và tính linh hoạt trong thiết kế hệ thống. Đây cũng là lí do thể loại này được lựa chọn làm nền tảng cho nhiều nghiên cứu liên quan đến trí tuệ nhân tạo trong game, đặc biệt là các mô hình AI ra quyết định chiến thuật .

Cơ chế hoạt động của hệ thống theo lượt :

Cơ chế Turn Based được xây dựng dựa trên một vòng lặp lượt chơi rõ ràng, mọi thông tin hay hành động đều được chuyển theo từng lượt thành một vòng trong đó mỗi lượt đại diện cho một chu kỳ hành động hoàn chỉnh của một nhân vật hoặc một nhóm nhân vật. Vòng lặp này thường bắt đầu bằng việc xác định thứ tự hành động dựa trên các yếu tố như tốc độ, chỉ số ưu tiên hoặc các quy tắc được định nghĩa trước. Sau khi thứ tự lượt được xác lập, Hệ thống sẽ lần lượt cho phép từng nhân vật thực hiện hành động của bản thân .[5]

Trong một lượt chơi, quá trình xử lý thương được chia thành nhiều giai đoạn nhằm đảm bảo tính nhất quán của hệ thống dễ thấy thì sẽ là ba trạng thái của một lượt , bắt đầu lượt trong lượt và sau lượt . Giai đoạn bắt đầu lượt có nhiệm vụ cập nhật các trạng thái ảnh hưởng lên nhân vật, chẳng hạn như hiệu ứng tăng hoặc giảm chỉ số. Tiếp theo là giai đoạn

hành động, nơi nhân vật có thể lựa chọn và thực hiện các hành vi như tấn công, phòng thủ hoặc sử dụng kỹ năng. Cuối cùng, giai đoạn kết thúc lượt sẽ xử lý các sự kiện còn lại và chuẩn bị chuyển sang lượt tiếp theo . [5]

Việc chia nhỏ lượt chơi thành các giai đoạn không chỉ giúp gameplay trở nên mạch lạc mà còn đóng vai trò quan trọng trong thiết kế trí tuệ nhân tạo. Ai có thể đưa ra quyết định dựa trên trạng thái ổn định của hệ thống tại từng giai đoạn, từ đó giảm thiểu các tình huống xung đột logic [2] . Cơ chế này cũng giúp việc mở rộng hệ thống chiến đấu trong tương lai mà không ảnh hưởng đến cấu trúc tổng thể của game.

Các thành phần cơ bản trong game Turn-Based :

Một game chiến thuật theo lượt thường bao gồm nhiều thành phần cốt lõi có mối quan hệ gắn kết với nhau. Trung tâm của hệ thống là các nhân vật tham gia chiến đấu, bao gồm nhân vật do người chơi điều khiển và nhân vật do AI điều khiển. Mỗi nhân vật được định nghĩa bởi tập hợp các chỉ số cơ bản như lượng Hp , sức tấn công ,khả năng phòng thủ và tốc độ, những yếu tố này quyết định khả năng hành động và vai trò chiến thuật của nhân vật trong trận đấu [5].

Bên cạnh hệ thống nhân vật, game Turn based còn bao gồm thêm các hệ thống như kỹ năng và hành động, cho phép nhân vật tương tác với môi trường và đối thủ. Các kỹ năng này thường được thiết kế để tạo ra sự đa dạng trong chiến thuật , khiến người và AI lựa chọn hành động phù hợp với tình huống. Ngoài ra còn, hệ thống trạng thái như hiệu ứng tăng cường hoặc suy yếu cũng góp phần làm phong phú gameplay, buộc người chơi phải cân nhắc nhiều yếu tố trước khi đưa ra quyết định.

Sự kết hợp giữa các thành phần trên tạo nên một hệ thống gameplay hoàn chỉnh trong đó mỗi yếu tố đều có vai trò nhất định trong việc định hình trải nghiệm chơi. Việc hiểu rõ các thành phần này là một việc rất quan trọng trong quá trình thiết kế và xây dựng hệ thống chiến đấu cũng như hệ thống AI trong các chương tiếp theo.

Đặc điểm và vai trò của game Turn Based trong thiết kế game :

Game Turn based có ưu điểm nổi bật là dễ mở rộng và dễ kiểm soát về mặt logic. Nhờ vào cấu trúc theo lượt rõ ràng , nên việc dễ dàng bổ sung thêm kỹ năng, nhân vật hoặc lượt chơi mới mà không làm ảnh hưởng đến toàn bộ hệ thống [5], Bên lại , nếu không đầu tư kỹ vào hệ thống trí tuệ nhân tạo của kẻ thù gameplay có thể trở nên lặp lại và thiếu tính thử

thách. Vì vậy , AI đóng vai trò đặc biệt quan trọng trong việc nâng cao chất lượng trải nghiệm người chơi đối với thể loại game này .

1.2 Trí Tuệ nhân Tạo trong Game :

Khái niệm và vai trò của trí tuệ nhân tạo trong Game :

Trí tuệ nhân tạo trong game được sử dụng nhằm mục đích là mô phỏng hành vi của nhân vật không do người chơi điều khiển, tạo cảm giác đối thủ có khả năng tư duy suy nghĩ và phản ứng hợp lý tùy tình huống [2].

Khác với trí tuệ nhân tạo trong lĩnh vực học máy, AI trong game chủ yếu tập trung vào việc tạo ra hành vi hợp lý, những hành vi này để dự đoán ở mức vừa phải nhưng vẫn phải đủ thử thách để duy trì sự hứng thú cho người chơi . Một hệ thống AI tốt sẽ đóng góp không nhỏ cho phần gameplay trở nên sống động và có chiều sâu.

Các mô hình AI phổ biến trong game :

Trong quá trình phát triển game, nhiều mô hình AI đã được sử dụng nhằm điều khiển hành vi của nhân vật. Các mô hình phổ biến bao gồm AI dựa trên các luật hardcoded (role-base-rule) [10], máy trạng thái hữu hạn (finite-state machine FSM) [11] và cây hành vi (Behavies tree) [11] . Mỗi mô hình đều có ưu điểm riêng nhưng cũng tồn tại những hạn chế nhất định, đặc biệt là về khả năng mở rộng và tính linh hoạt khi số lượng hành động tăng lên. Điều này đặt ra yêu cầu cần có một mô hình AI phù hợp hơn cho các game chiến thuật theo lượt có nhiều lựa chọn hành động .

Các mô hình AI truyền thống gặp khó khăn khi xử lý những tình huống phức tạp và thay đổi liên tục dựa trên hành động của người chơi trong gamelay. AI dựa trên role-base-rule dễ trở nên cứng nhắc và cực khó bảo trì sửa chữa nâng cấp khi số lượng luật được tăng thêm ,AI dựa trên thuật toán FSM rất dễ cài đặt nhưng nhanh chóng trở nên phức tạp khi số lượng trạng thái lớn nên có điểm yếu giống với role based .Cây hành vi có khả năng mô tả hành vi chi tiết nhưng đòi hỏi cấu trúc rất phức tạp và rất khó mở rộng. Những hạn chế này khiến việc áp dụng các mô hình AI truyền thống vào game turn based trở nên đắn đo với từng loại để tối ưu hiệu suất [2] [10] .

1.3 Mô hình Utility AI :

1.3.1 Nói sơ về mô hình USS (Utility Scoring System) :

Utility Scoring System AI là một hệ thống mô hình trí tuệ nhân tạo dựa trên việc đánh giá mức độ hữu ích của từng loại hành động trong một tình huống cụ thể nhất định.

Thay vì lựa chọn hành động theo luật cố định Utility AI tính toán điểm số cho mỗi hành động dựa trên trạng thái hiện tại của game. Theo Dave Mark, Utility AI là mô hình ra quyết định trong đó mỗi hành động được đánh giá thông qua một giá trị utility phản ánh mức độ phù hợp của hành động với trạng thái hiện tại của hệ thống.[12] Cách tiếp cận này giúp AI đưa ra quyết định linh hoạt và phù hợp với từng hoàn cảnh cụ thể trong trận đấu dựa trên hành động có utility cao nhất.

Một hệ thống Utility AI thường bao gồm các hành động, các yếu tố để đánh giá và hàm tính điểm. Mỗi hành động được gán một tập các yếu tố đánh giá dựa trên dữ liệu đầu vào như Hp (health point) mức độ nguy hiểm hoặc trạng thái của đối thủ. Các yếu tố này được đưa vào hàm đánh giá rồi tính ra điểm số cuối cùng sau đó sẽ ra được quyết định cho hành động. Hành động có điểm cao nhất sẽ được AI chọn lựa để thực hiện trong lượt hiện tại [9] .

Quy trình ra quyết định của Utility AI bắt đầu bằng việc thu thập dữ liệu trạng thái của game , người chơi tại thời điểm đến lượt của AI [6]. Sau đó ,hệ thống tiến hành tính điểm số cho từng hành động dựa trên các hàm đánh giá đã được thiết kế trước, cuối cùng AI so sánh các điểm số này và lựa chọn hành động có giá trị cao nhất để thực hiện[12]. Quy trình này được lặp lại ở mỗi lượt, giúp AI thích nghi với diễn biến của trận đấu .

Utility AI mang lại nhiều lợi ích khi nghiên cứu và áp dụng vào game chiến thuật theo lượt. Mô hình này cho phép AI đưa ra quyết định linh hoạt, tránh lặp lại hành vi và dễ dàng mở rộng khi bổ sung thêm hành động mới. Ngoài ra, Utility Ai còn giúp người phát triển tinh chỉnh hành vi của AI chỉ thông qua việc điều chỉnh các hàm đánh giá hoặc các giá trị preset mà không cần thay đổi toàn bộ hệ thống logic.

Công thức thông thường của một Utility AI thường có cách biểu đạt như thế này [6]:

$$Utility = \sum(Consideration_i \times Weight_i)$$

Trong đó :

Consideration : yếu tố đánh giá

Weight : trọng số

Utility : điểm cuối cùng

Dựa trên công thức trên thì đây sẽ là một ví dụ cụ thể cô đồ án với trường hợp là tính utility score dựa trên Hp (Heath point) của người chơi :

$$\text{Utility} = (\text{HPScore} \times \text{hpBias}) + (\text{ThreatScore} \times \text{threatBias}) \\ + (\text{FinisherScore} \times \text{finisherWeight} + \text{Noise})$$

Bây giờ công thức trên sẽ được phân tích như sau , hành động utility được tính bằng cách tính tổng của (phần trăm tổng Hp của người chơi nhân với trọng số về Hp) + (Điểm số mức độ quan trọng/nguy hiểm của người chơi nhân với ưu tiên điểm trọng yếu của người chơi) + (phần trăm Hp còn lại nhân với trọng số kết liễu) + độ ngẫu nhiên sau khi tính xong thì sẽ chọn được mục tiêu để hành động .

1.3.2 Lý do chọn Utility AI cho game turn based :

Game chiến thuật theo lượt thường cung cấp cho nhân vật nhiều lựa chọn hành động khác nhau trong mỗi lượt, chẳng hạn như tấn công phòng thủ, sử dụng kỹ năng hoặc hỗ trợ đồng đội , khi số lượng hành động tăng lên thì việc sử dụng các mô hình AI dựa trên luật hoặc trạng thái hữu hạn sẽ trở nên lèp vế hăn và trở nên kém hiệu quả do độ phức tạp tăng nhanh và khó bảo trì [1].

Utility Ai giải quyết vấn đề này bằng cách cho phép tất cả các hành động được đánh giá song song dựa trên cùng một tập dữ liệu trạng thái. Mỗi hành động được xem như một sự lựa chọn đơn lẻ, có thể được thêm mới chỉnh sửa tùy ý mà không ảnh hưởng quá nhiều đến hệ thống cấu trúc tổng thể của cả hệ thống AI . Giúp việc mở rộng và tinh chỉnh trở nên đơn giản và linh hoạt hơn [6].

Theo Dave Mark và Kevin Dill [1] , Utility AI đặc biệt phù hợp với các trò chơi có số lượng hành động lớn và nhiều yếu tố ảnh hưởng đến quá trình ra quyết định .Mô hình này cho phép mở rộng nhanh chóng đơn gian thông qua việc chỉnh sửa thay đổi các hàm đánh giá và trọng số mà không cần sửa đổi cấu trúc tổng thể của hệ thống .

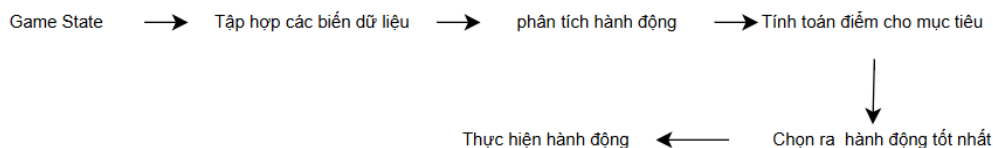
Thể loại turn based RPG ,thì AI chỉ cần đưa ra quyết định tại các thời điểm xác định trong mỗi lượt chơi, tạo điều kiện thuận lợi cho việc thực hiện các phép tính đánh giá và đưa ra quyết định mà không ảnh hưởng đến hiệu năng hay trải nghiệm của người chơi.

1.3.3 Cấu trúc tổng thể của hệ thống Utility :

Trong hệ thống được xây dựng trong đề tài, mỗi lượt hành động của AI bắt đầu bằng việc thu thập thông tin từ trạng thái hiện tại của trận đấu như lượng Hp của các nhân vật , mức độ nguy hiểm (threat), trạng thái hiệu ứng và số lượng mục tiêu còn sống .

Các dữ liệu này được sử dụng để tính toán Utility Score cho từng hành động khả thi. Mỗi hành động được đánh giá thông qua một tập các yếu tố đặc trưng như khả năng gây sát thương, khả năng kết liễu mục tiêu , Hp của đồng minh .Sau toàn bộ các điểm utility đã tính toán thì hệ thống sẽ chọn hành động có số điểm cao nhất hay best scored

Hệ thống đưa ra hành động dựa trên best score với cấu trúc như thế AI ra lựa chọn rất linh hoạt, dễ mở rộng và điều chỉnh hành vi thông qua các hàm đánh giá mà không cần can thiệp vào cấu trúc lõi của logic tổng thể .



Hình 1:Luồng dữ liệu biến utility

1.3.4 Ưu điểm của hệ thống Utility :

Utility mang lại nhiều ưu điểm nổi bật khi áp dụng vào game chiến thuật theo lượt . mô hình này cho phép AI đưa ra các quyết định linh hoạt, khả năng thích nghi cao với nhiều tình huống khác nhau . Việc thêm mới hoặc chỉnh sửa không ảnh hưởng đến cấu trúc hoặc logic tổng thể giúp việc mở rộng hay bảo trì trở nên đơn giản .

Ngoài ra hệ thống cũng khiến cho hành vi của các kẻ thù trong game ít bị lặp lại nhờ khả năng đánh giá hành động dựa trên nhiều yếu tố, AI có thể đưa ra nhiều kết quả khác nhau ngay cả trong những tình huống giống nhau giúp tăng trải nghiệm.

Dựa trên tài liệu [2] [6] thì đại khái ta có một bản so sánh đơn giản :

Hình 2 : bảng so sánh các mô hình

Tiêu chí	Rule-Based	FSM	Utility AI
Mở rộng hành động	Thấp	Cao	Cao
Linh hoạt	Thấp	Khá	Cao
Dễ cân bằng	Cao	Khá	Cao
Phù hợp với turn based RPG	Khá	Cao	Cao

1.4 Thiết kế kiến trúc hệ thống game :

1.4.1 Kiến trúc tổng thể của hệ thống game :

Để đảm bảo khả năng mở rộng, bảo trì và phát triển thì hệ thống game được thiết kế theo mô hình phân tách chức năng (Modular Architecture), trong đó mỗi thành phần đảm nhận và quản lý một nhiệm vụ riêng biệt và giao tiếp với nhau qua các lớp quản lý trung gian. Kiến trúc tổng thể của hệ thống bao gồm các thành phần chính là :

Hệ thống quản lý dữ liệu (Data management System).

Hệ thống quản lý nhân vật và kỹ năng .

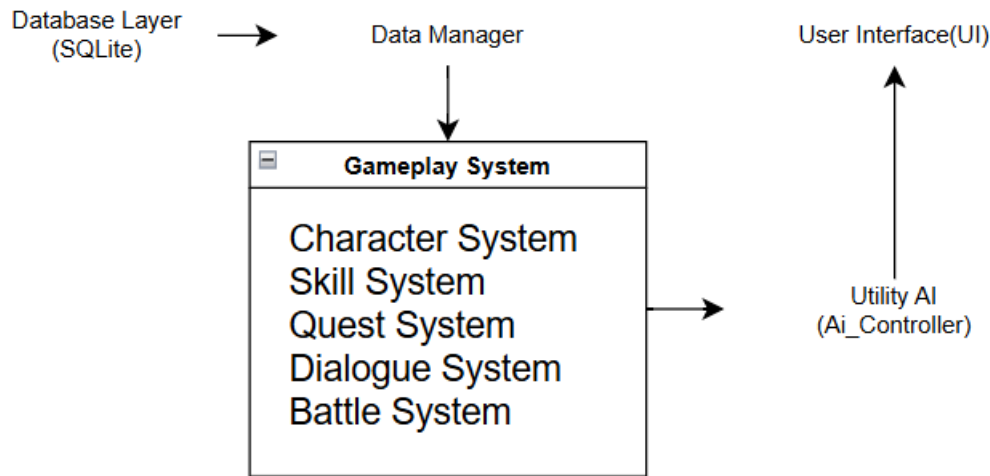
Hệ thống chiến đấu theo lượt (Battle System)

Hệ thống quản lý trí tuệ nhân tạo (Utility AI System).

Hệ thống giao diện người dùng (UI System).

Trong khối kiến trúc này thì dữ liệu nhân vật, kỹ năng ,Item ,hội thoại ,...các thứ khác đều được lưu trữ trong cơ sở dữ liệu SQLite và được truy xuất thông qua các lớp quản lý dữ liệu đã được xử lý thay vì truy cập trực tiếp vào cơ sở dữ liệu giúp giảm phụ thuộc tăng khả năng mở rộng .

Hệ thống Ai được thiết kế tách biệt hoàn toàn khỏi phần thực thi gameplay. Ai chỉ có nhiệm vụ phân tích trạng thái trận đấu và đưa ra lựa chọn trong lượt hành động. Quá trình thực thi kỹ năng tính toán sát thương và cập nhật trạng thái nhân vật sẽ được thực thi xử lý bởi Battle System (gồm battleManager , Battle Unit) các tiếp cận giúp việc nâng cấp sửa chữa thay thế đều không đụng chạm đến các thành phần còn lại của trò chơi.



Hình 3: Sơ đồ luồng xử lý dữ liệu

1.4.2 Thiết kế hệ thống chiến đấu theo lượt :

Hệ thống chiến đấu theo lượt là trung tâm của gameplay và đóng vai trò quan trọng trong việc triển khai các cơ chế chiến thuật. Hệ thống này chịu trách nhiệm quản lý thứ tự lượt, xử lý hành động của nhân vật và cập nhật trạng thái trận đấu sau mỗi lượt, việc thiết kế hệ thống chiến đấu cần đảm bảo tính chính xác và minh bạch để người chơi có thể dễ dàng theo dõi và đưa ra quyết định chiến thuật.

Trong đề tài này, hệ thống chiến đấu được thiết kế theo hướng tách biệt logic xử lý và phần hiển thị, giúp giảm sự phụ thuộc giữa lõi chơi và giao diện người dùng. Cách tiếp cận này giúp giảm tải công việc khi bảo trì và đánh giá hiệu quả của hệ thống AI trong các tình huống chiến đấu khác nhau.

Quy trình xử lý một lượt chiến đấu bao gồm các giai đoạn theo thứ tự sau :

1. Xác định nhân vật được phép hành động.
2. Cập nhật hiệu ứng trạng thái đầu lượt.
3. Lựa chọn hành động.
4. Thực thi kỹ năng hoặc đòn đánh.
5. Tính toán sát thương và hiệu ứng.
6. Cập nhật trạng thái nhân vật.
7. Kiểm tra điều kiện thắng thua.
8. Chuyển sang lượt tiếp theo.

1.4.3 Thiết kế hệ thống trí tuệ nhân tạo :

Hệ thống được thiết kế trong đề được xây dựng dựa trên mô hình Utility System. Thay vì sử dụng các tập luật cố định hoặc các trạng thái hữu hạn, AI đánh giá đồng thời tất cả các hành động khả thi và lựa chọn hoành thành hành động được tính có giá trị cao nhất.

Để tạo ra các phong cách chiến đấu khác nhau, hệ thống sử dụng Utility Profile. Mỗi loại kẻ địch sẽ được gán cho một bộ trọng số riêng cho các hành vi tấn công , hồi máu phòng thủ, gây hiệu ứng bất lợi hoặc truy sát người chơi

Ví dụ :

Aggressive: với Hp bias, finished weight cao ưu tiên gây sát thương cho người chơi

Healer : với ally low Hp weight , kiểm tra bản thân có skill trị liệu đang hoạt động để hồi phục cho đồng minh có Hp thấp phe mình .

Nemesis: bỏ qua các chỉ số khác tập trung vào threat , tấn công mục tiêu có threat cao nhất .

Ngoài hệ thống utility cơ bản , AI trong đề tài còn được hỗ trợ thêm một hệ thống nữa là cơ chế Threat system để đánh giá mức độ nguy hiểm cho từng thành viên của người chơi dựa trên lượng sát thương mà nhân vật đó gây ra , lượng hồi phục mà nhân vật mang lại . Kết quả đánh giá này được thêm vào trong quá trình chọn mục tiêu của utility AI tạo ra các hành vi hợp lý hơn .

Việc áp dụng Utility Ai giúp hệ thống đạt được khả năng mở rộng cao dễ dàng bổ sung hành động mới và tạo ra phong cách chiến đấu khác nhau mà không cần lập trình nhiều thuật toán AI riêng

1.5 Công nghệ và công cụ sử dụng :

1.5.1 Unity Engine :

Unity là một trong những công cụ phát triển game phổ biến hiện nay, hỗ trợ tốt cho việc xây dựng game chiến thuật theo lượt. Với kiến trúc dựa trên Component, Unity cho phép nhà phát triển tổ chức mã nguồn một cách linh hoạt và dễ mở rộng . Điều này đặc biệt

phù hợp với các dự án nghiên cứu và phát triển game ở mức prototype, nơi yêu cầu thay đổi và thử nghiệm hệ thống diễn ra thường xuyên.

Unity còn cung cấp hệ thống sinh tái phong phú với nhiều công cụ hỗ trợ quản lý scene, xử lý sự kiện và xây dựng giao diện người dùng. Những đặc điểm này giúp quá trình triển khai gameplay và hệ thống AI trở nên hiệu quả, rút ngắn thời gian phát triển hơn .[7]

1.5.2 Component :

Unity được xây dựng trên các kiến trúc lập trình theo hướng component, trong đó mỗi đối tượng trong game được cấu thành từ nhiều thành phần chức năng độc lập gọi là component, thay vì sử dụng cách kế thừa theo mô hình phân cấp cứng nhắc, Unity cho phép nhà phát triển gắn các component khác nhau lên một đối tượng để mở rộng hoặc thay đổi hành vi của đối tượng đó trong quá trình phát triển.[7]

Trong đề tài này , các mô hình component được ứng dụng rộng rãi để xây dựng các thực thể trong game như nhân vật , kẻ địch và các đối tượng tương tác trong môi trường, mỗi nhân vật được thành từ nhiều component riêng biệt như component quản lý các chỉ số, component quản lý hành động chiến đấu và component điều khiển AI còn rất nhiều loại nữa ,... Việc tách biệt các chức năng thành các component độc lập giúp giảm sự phụ thuộc giữa các phần của hệ thống đồng thời giúp quá trình kiểm thử và bảo trì dễ dàng .[7]

1.5.3 Prefab :

Prefab là một trong những cơ chế quan trọng nhất trong Unity, cho phép lưu trữ và tái sử dụng các đối tượng gameobject đã được cấu hình sẵn. Một prefab có thể bao gồm một hoặc nhiều gameobject cùng với toàn bộ các component, giá trị thuộc tính và mối quan hệ con và cha của chúng , khi một prefab được tạo ra nó trở thành cái khuôn mẫu để nhân bản thêm nhiều lại bản khác nhau trong game dựa theo nhu cầu .[8]

Việc sử dụng prefab giúp đảm bảo tính nhất quán giữa các đối tượng cùng loại và giảm đáng kể thời gian phát triển. Khi chỉnh sửa file prefab gốc, mọi bản thể của prefab đó có trong dự án và các scene khác nhau đều được cập nhật đồng bộ, điều này rất hữu ích trong các dự án game có quy mô lớn hoặc có nhiều đối tượng lặp lại như nhân vật , kẻ địch hoặc các điểm kích hoạt sự kiện .

Trong phạm vi đề tài, prefab được sử dụng để xây dựng các thực thể chiến đấu, các điểm chạm kích hoạt trận đánh và các thành phần nằm trên phần giao diện người dùng. Các prefab encounter trong game đóng vai trò kích hoạt battle scene và được thiết kế để có thể dễ dàng bật hoặc tắt tùy ý theo trạng thái gameplay. Cách sử dụng prefab giúp đảm bảo luồng gameplay diễn ra liền mạch và nhất quán, đồng thời làm giảm trọng tải cho công việc mở rộng về sau.[8]

Scriptable object và vai trò trong quản lý dữ liệu :

Scriptable Object là một kiểu đối tượng đặc biệt trong Unity được sử dụng để lưu trữ dữ liệu độc lập với các đối tượng trong Scene. Khác với MonoBehaviour, Scriptable Object không cần gắn trực tiếp vào GameObject mà có thể tồn tại dưới dạng các tệp tài nguyên (Asset), cho phép nhiều đối tượng cùng truy cập và sử dụng chung một nguồn dữ liệu.

Trong đề tài này, Scriptable Object được sử dụng để quản lý các dữ liệu cấu hình như thông tin nhân vật, kỹ năng, vật phẩm, hiệu ứng trạng thái và các tham số của hệ thống AI. Việc tách biệt dữ liệu khỏi mã nguồn giúp quá trình chỉnh sửa, cân bằng gameplay và mở rộng nội dung game trở nên thuận tiện hơn mà không cần thay đổi logic chương trình.[8]

Ngôn ngữ lập trình C# :

C# là ngôn ngữ lập trình chính được sử dụng trong Unity, hỗ trợ lập trình hướng đối tượng và quản lý mã nguồn hiệu quả. Việc sử dụng C# giúp xây dựng các hệ thống game có cấu trúc rõ ràng, dễ đọc và dễ bảo trì, phù hợp với yêu cầu của một sản phẩm game chiến thuật có quy mô mở rộng.

Trong đề tài này, C# được sử dụng để xây dựng toàn bộ các hệ thống cốt lõi như quản lý nhân vật, hệ thống chiến đấu theo lượt, trí tuệ nhân tạo Utility AI, quản lý dữ liệu, giao diện người dùng và các công cụ hỗ trợ phát triển trong Unity Editor. Nhờ khả năng tích hợp chặt chẽ với Unity cùng hệ sinh thái thư viện phong phú, C# giúp việc triển khai các chức năng diễn ra thuận lợi, đồng thời hỗ trợ mở rộng và bảo trì hệ thống trong các giai đoạn phát triển tiếp theo.

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1 Mục tiêu và vai trò của giai đoạn phân tích thiết kế :

Giai đoạn phân tích và thiết kế đóng vai trò quan trọng trong quá trình phát triển hệ thống game, đặc biệt đối với các dự án có tính nghiên cứu và tích hợp nhiều thành phần chức năng như hệ thống chiến đấu, trí tuệ nhân tạo và quản lý dữ liệu.

Theo Tracy Fullerton[1], việc xác định rõ yêu cầu và kiến trúc hệ thống ngay từ đầu giúp giảm thiểu rủi ro trong quá trình phát triển .

Đối với đề tài phát triển game nhập vai chiến thuật theo lượt kết hợp trí tuệ nhân tạo, giai đoạn này càng trở nên quan trọng do hệ thống có nhiều thành phần phức tạp như quản lý lượt chơi, xử lý hành động chiến đấu và điều phối hành vi AI. Việc thiết kế rõ ràng giúp đảm bảo rằng các thành phần gameplay và AI có thể hoạt động độc lập nhưng vẫn phối hợp nhịp nhàng trong toàn bộ hệ thống.

Trong đề tài này, giai đoạn phân tích và thiết kế tập trung vào ba mục tiêu chính. Thứ nhất là xác định rõ các yêu cầu chức năng và phi chức năng của game. Thứ hai là xây dựng kiến trúc hệ thống phù hợp với mô hình game chiến thuật theo lượt và Utility AI. Cuối cùng là thiết kế chi tiết các hệ thống cốt lõi, làm cơ sở cho việc triển khai và đánh giá ở các chương tiếp theo.

2.2. Phân tích yêu cầu hệ thống

2.2.1. Đối tượng sử dụng và bối cảnh hệ thống

Hệ thống game được xây dựng hướng đến đối tượng là nhóm người chơi phổ thông có yêu thích thể loại chiến thuật theo lượt và đặc biệt là các người chơi có xu hướng trải nghiệm cốt truyện và xây dựng chiến thuật thông qua việc quản lý đội hình nhân vật. Người chơi sẽ điều khiển tổ đội gồm ba thành viên khác nhau, sử dụng kỹ năng và phối hợp chiến thuật để đánh bại hệ thống AI điều khiển.

Bối cảnh hệ thống được xây dựng dưới dạng một game đơn, không yêu cầu kết nối mạng và không có tính năng multiplayer. Điều này giúp tập trung toàn bộ nguồn lực vào việc xây dựng gameplay chiến đấu và hệ thống AI, phù hợp với phạm vi của một dự án

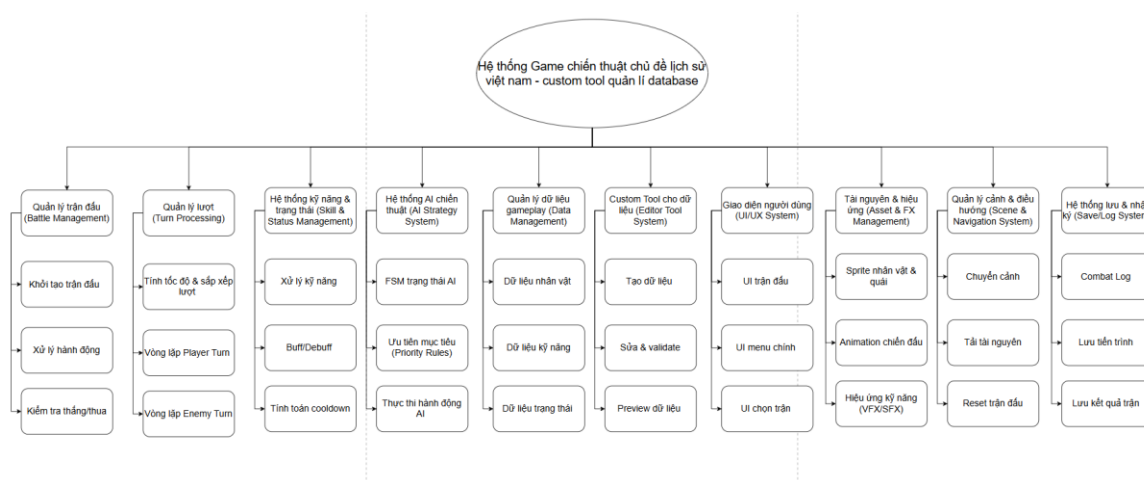
thực tập tốt nghiệp. Game vận hành theo mô hình chuyển đổi giữa các scene, bao gồm scene cốt truyện và scene chiến đấu, tạo thành một luồng gameplay hoàn chỉnh.

2.2.2. Yêu cầu chức năng của hệ thống

Hệ thống cần đáp ứng đầy đủ các chức năng cơ bản của một game nhập vai chiến thuật theo lượt. Người chơi phải có khả năng quản lý đội hình nhân vật, tham gia chiến đấu, sử dụng kỹ năng, tương tác với các NPC, nhận nhiệm vụ và theo dõi tiến trình phát triển của nhân vật trong suốt quá trình chơi.

Trong hệ thống chiến đấu, game phải quản lý chính xác thứ tự lượt hành động giữa các đơn vị tham gia chiến đấu. Mỗi nhân vật cần có khả năng thực hiện các hành động như tấn công thường, sử dụng kỹ năng, phòng thủ hoặc sử dụng vật phẩm. Hệ thống đồng thời phải xử lý sát thương, hiệu ứng trạng thái, hồi máu và các sự kiện chiến đấu khác theo đúng logic được thiết kế.

Đối với hệ thống AI, các nhân vật do AI điều khiển phải có khả năng tự động đánh giá tình huống chiến đấu và đưa ra các quyết định phù hợp. AI cần thực hiện việc lựa chọn hành động, lựa chọn mục tiêu và thay đổi chiến thuật dựa trên trạng thái hiện tại của trận đấu. Quá trình ra quyết định được xây dựng dựa trên mô hình Utility Scoring System nhằm tạo ra các hành vi tối ưu và hạn chế lặp hành động trong gameplay [2][6][9].



Hình 4 : Sơ đồ phân rã chức năng

Ngoài ra, hệ thống cần cung cấp các công cụ hỗ trợ quản lý dữ liệu nhân vật, kỹ năng, nhiệm vụ và hội thoại thông qua các Custom Tool được xây dựng trong Unity Editor.

Các công cụ này giúp giảm thời gian nhập liệu thủ công, tăng tính nhất quán của dữ liệu và hỗ trợ quá trình phát triển nội dung game.

2.2.3. Yêu cầu phi chức năng

Ngoài các yêu cầu chức năng, hệ thống game cũng cần đáp ứng một số yêu cầu phi chức năng nhằm đảm bảo chất lượng và khả năng phát triển lâu dài. Trước hết, hệ thống cần có kiến trúc rõ ràng, dễ hiểu và dễ mở rộng. Điều này cho phép bổ sung thêm nội dung gameplay hoặc nâng cấp hệ thống AI trong tương lai mà không cần thay đổi lớn cấu trúc hiện tại.

Bên cạnh đó, hiệu năng của hệ thống phải đảm bảo ở mức ổn định, đặc biệt trong các trận chiến có nhiều đơn vị tham gia. Mặc dù game chỉ dừng ở mức prototype, việc thiết kế hợp lý vẫn giúp hạn chế các vấn đề về hiệu suất và đảm bảo trải nghiệm người chơi không bị gián đoạn.

Cuối cùng, hệ thống cần có tính dễ bảo trì và dễ kiểm thử. Việc tách biệt rõ ràng giữa logic gameplay, hệ thống AI và giao diện người dùng giúp quá trình kiểm thử từng thành phần diễn ra thuận lợi hơn, đồng thời hỗ trợ việc phát hiện và sửa lỗi hiệu quả.

2.3. Phân tích luồng hoạt động của hệ thống

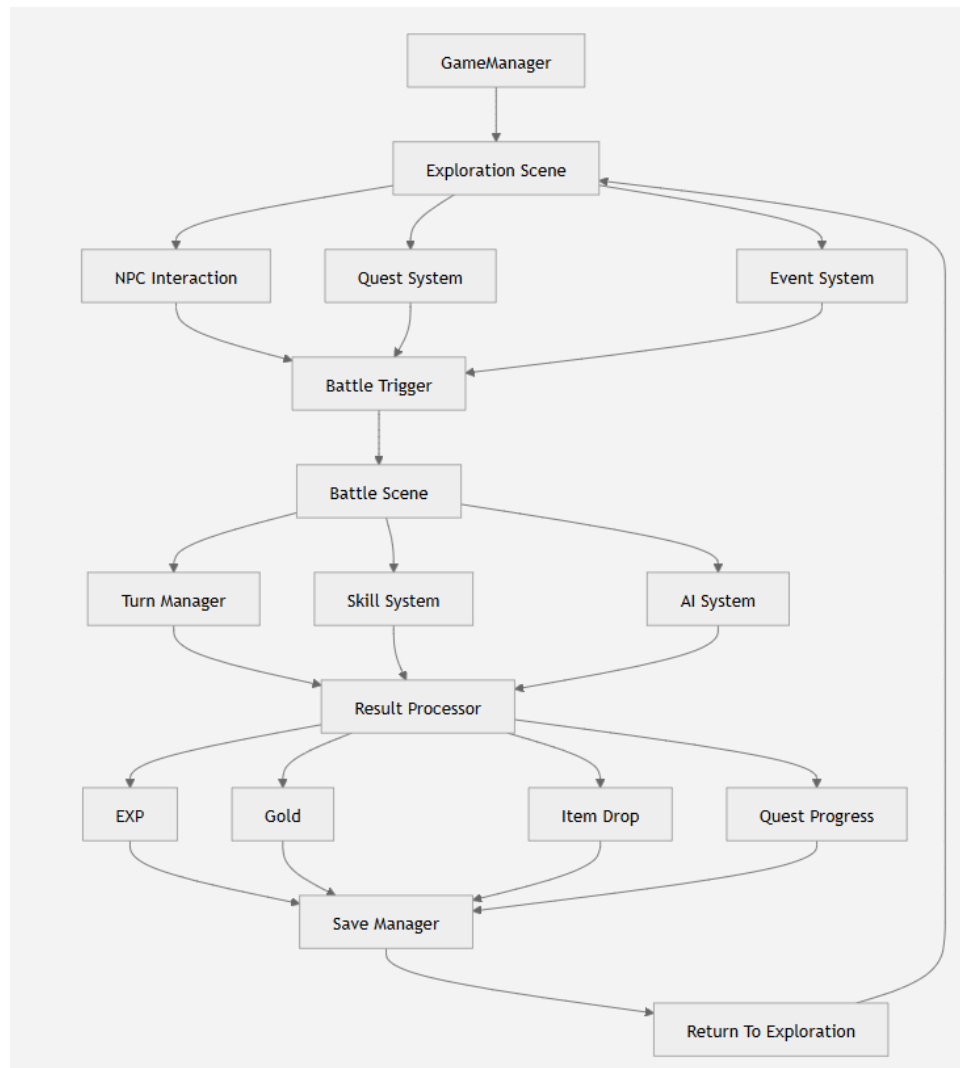
2.3.1. Luồng tổng quát của gameplay

Luồng hoạt động tổng quát của hệ thống game được thiết kế xoay quanh hai trạng thái chính là trạng thái khám phá cốt truyện và trạng thái chiến đấu. Trong trạng thái khám phá, người chơi điều khiển nhân vật di chuyển trong môi trường và tương tác với các đối tượng trong scene. Khi người chơi kích hoạt một sự kiện chiến đấu, hệ thống sẽ chuyển sang trạng thái chiến đấu thông qua việc tải scene tương ứng.

Khi người chơi tiếp xúc với một sự kiện chiến đấu hoặc kích hoạt một điều kiện đặc biệt, hệ thống sẽ lưu lại trạng thái hiện tại của nhân vật và chuyển sang battle scene, tại đây mọi chỉ số của nhân vật, thông tin đội hình, các trang bị đi kèm và thông tin của trận đấu được khởi tạo để bắt đầu quá trình chiến đấu.

Sau khi trận đấu kết thúc, hệ thống xác định kết quả thắng thua, cập nhật dữ liệu liên quan đến kinh nghiệm, nhiệm vụ hoặc trạng thái cốt truyện, sau đó chuyển người chơi lại

về scene khám phá (story scene). Quá trình này tạo thành một vòng lặp gameplay liên tục giữa khám phá , chiến đấu , tinh chỉnh nhân vật .[1][5]



Hình 5 : Sơ đồ Luồng dữ liệu

2.3.2. Luồng xử lý trong battle scene

Khi Battle scene được khởi tạo ,hệ thống tiến hành nạp dữ liệu đội hình của người chơi và kẻ địch từ cơ sở dữ liệu sau đó các đơn vị chiến đấu được sinh ra tại các vị trí tung úng và hệ thống bắt đầu thiết lập trạng thái ban đầu của trận đấu .

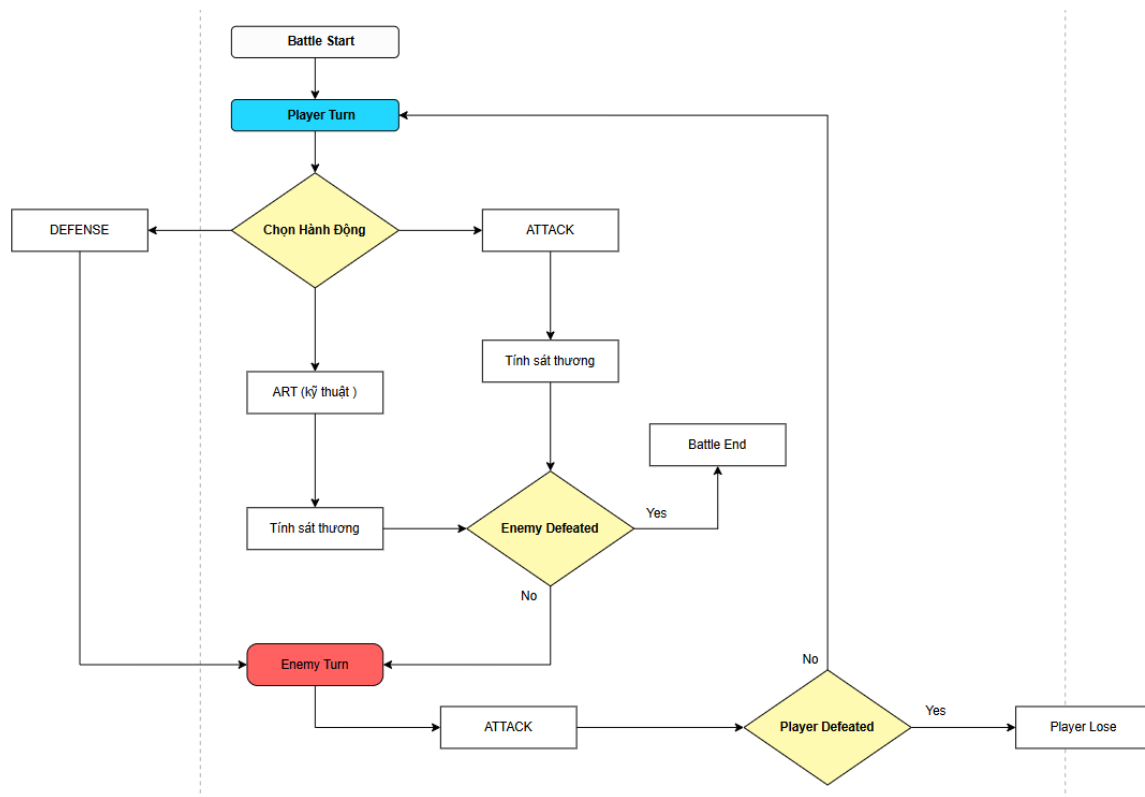
Toàn bộ trận chiến được điều khiển bởi Battle Manager thông qua cơ chế xử lý theo lượt ,mỗi lượt được chia thành các giai đoạn nhằm đảm bảo tính nhất quán.[7]

Hệ thống hàng chờ lượt :

Khởi tạo lượt , cập nhật hiệu ứng trạng thái , xác định hành động , thực thi hành động cập nhật dữ liệu trận đấu, kiểm tra điều kiện thắng thua , chuyển sang lượt kế tiếp.[8]

Đối với nhân vật của người chơi, hệ thống hiển thị các lựa chọn hành động thông qua giao diện người dùng bao gồm đánh thường thực hiện kỹ năng sử dụng item. Sau khi hành động được xác nhận, hệ thống thực thi hành động và cập nhật trạng thái trận đấu.

Đối với hành động của nhân vật do AI điều khiển, hệ thống battle manager chuyển quyền xử lý sang mô-đun Utility AI ,kết quả đánh giá từ AI sẽ được chuyển lại cho hệ thống chiến đấu và trả về hành động tối ưu dựa trên mô hình Utility AI.[1]



Hình 6 : Sơ đồ tuần tự

2.3.3. Luồng xử lý của hệ thống AI

Hệ thống AI được xây dựng dựa trên mô hình Utility Scoring System, trong đó mỗi lượt hành động của AI được xử lý thông qua một chuỗi các bước đánh giá và lựa chọn hành động .[9]

Khi đến lượt của một đơn vị AI, hệ thống trước tiên thu thập toàn bộ thông tin cần thiết từ trạng thái hiện tại của trận đấu. Các dữ liệu được sử dụng bao gồm lượng máu của đồng minh và kẻ địch, các hiệu ứng trạng thái đang tồn tại, danh sách kỹ năng có thể sử dụng, mức độ đe dọa (Threat Level) của từng mục tiêu và Utility Profile được gán cho đơn vị AI.[2]

Sau khi thu thập dữ liệu, hệ thống tiến hành đánh giá tất cả các hành động khả thi. Mỗi hành động được tính toán theo Utility Score thông qua các thành phần như Attack Score, Heal Score, Defend Score, Debuff Score, Finisher Score, Threat Score.[6][11]

Cách tiếp cận này giúp AI có khả năng thích nghi với nhiều tình huống chiến đấu khác nhau, tạo ra nhiều phong cách hành vi riêng biệt thông qua hệ thống Utility Profile mà không cần xây dựng nhiều thuật toán AI độc lập.

2.3.4. Phân tích cơ chế Threat System và Utility Profile:

Để nâng cao khả năng ra quyết định của AI trong chiến đấu, hệ thống không chỉ sử dụng Utility Score để đánh giá hành động mà còn kết hợp cơ chế Threat System và Utility Profile nhằm tạo ra nhiều phong cách chiến đấu khác nhau cho từng loại kẻ địch.

Threat System được sử dụng để đánh giá mức độ nguy hiểm của từng nhân vật người chơi đối với AI. Mỗi nhân vật sẽ sở hữu một giá trị Threat phản ánh mức độ ảnh hưởng của họ đến trận đấu. Giá trị này có thể được hình thành từ các yếu tố như lượng sát thương gây ra, khả năng hỗ trợ đồng đội hoặc các hành động có tác động lớn đến cục diện trận chiến.[2]

Trong quá trình lựa chọn mục tiêu, AI sẽ sử dụng Threat như một yếu tố đánh giá bổ sung thay vì chỉ dựa vào lượng máu hiện tại của đối phương. Điều này giúp AI có xu hướng ưu tiên tấn công các mục tiêu có khả năng gây nguy hiểm lớn thay vì lựa chọn ngẫu nhiên.

Ví dụ, nếu một nhân vật liên tục gây sát thương lớn cho đội hình kẻ địch, giá trị Threat của nhân vật đó sẽ tăng lên. Các đơn vị AI sở hữu trọng số Threat cao sẽ có xu hướng tập trung tấn công mục tiêu này nhằm giảm áp lực lên đội hình của chúng.

Thay vì xây dựng nhiều thuật toán AI khác nhau cho từng loại kẻ địch, hệ thống sử dụng một lõi Utility AI thống nhất kết hợp với Utility Profile.[6]

Utility Profile là tập hợp các trọng số được sử dụng trong quá trình tính Utility Score. Mỗi loại kẻ địch sẽ được gán một Profile riêng, từ đó tạo ra các hành vi chiến đấu khác nhau mặc dù vẫn sử dụng chung một thuật toán AI.

Ví dụ như Aggressive : ưu tiên gây sát thương ,

Healer : ưu tiên hồi máu và hỗ trợ đồng đội,

Assassin : ưu tiên kết liễu mục tiêu có hp hiện tại thấp .

Nemesis : ưu tiên mục tiêu có threat cao nhất.

Mỗi Profile được xây dựng từ các trọng số như Attack Weight, Heal Weight ,Defend Weight ,Debuff Weight ,AoE Weight ,Finisher Weight ,Threat Bias ,HP Bias. Các trọng số này được sử dụng để điều chỉnh cách AI đánh giá hành động và lựa chọn mục tiêu.[6]

Ví dụ đối với Profile Healer, trọng số Heal Weight được đặt cao hơn đáng kể so với Attack Weight. Khi phát hiện đồng minh có lượng máu thấp, điểm Utility của hành động hồi máu sẽ vượt qua các hành động khác, khiến AI ưu tiên hỗ trợ thay vì tấn công.

Ngược lại, Profile Assassin sở hữu Finisher Weight cao, giúp AI tập trung tìm kiếm các mục tiêu có lượng máu thấp để nhanh chóng loại bỏ khỏi trận đấu.

Cách tiếp cận này cho phép tạo ra nhiều phong cách chiến đấu khác nhau chỉ bằng việc thay đổi dữ liệu cấu hình, không cần xây dựng thêm thuật toán AI mới.

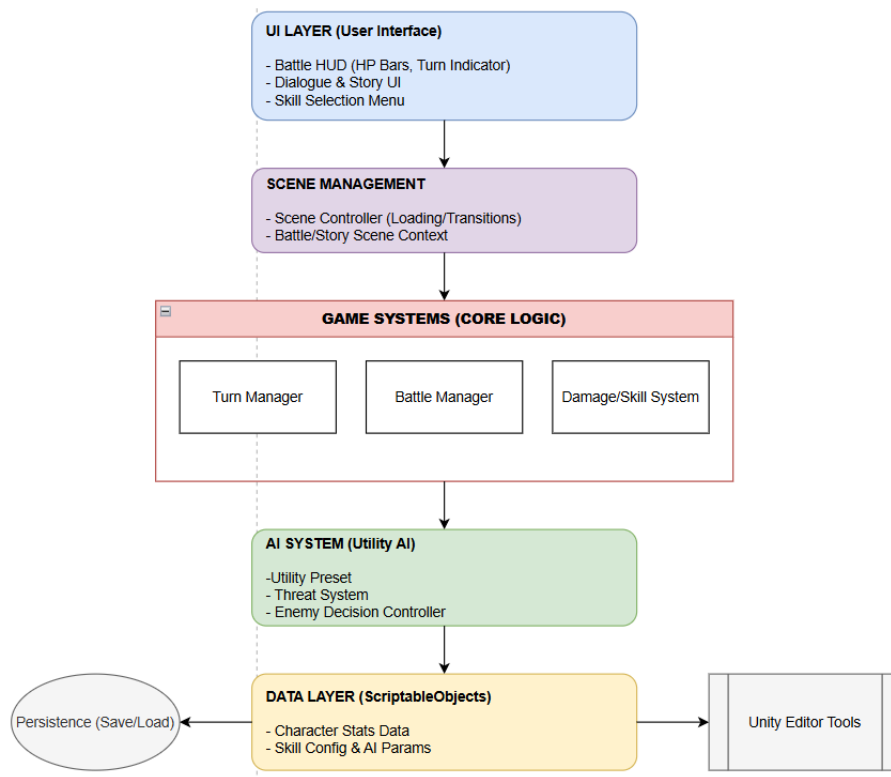
Sự kết hợp giữa Threat System và Utility Profile giúp AI đưa ra các quyết định mang tính chiến thuật hơn so với các mô hình AI truyền thống. Threat System giúp AI xác định đúng mục tiêu cần ưu tiên, trong khi Utility Profile quyết định phong cách chiến đấu của từng đơn vị.[9]

Nhờ đó, các kẻ địch trong game không chỉ hành động hợp lý mà còn thể hiện được vai trò riêng trong đội hình chiến đấu, góp phần nâng cao tính đa dạng và chiều sâu chiến thuật của gameplay.

2.4. Thiết kế kiến trúc hệ thống chi tiết

2.4.1. Nguyên tắc thiết kế kiến trúc hệ thống

Kiến trúc hệ thống trong đề tài này được thiết kế dựa trên các nguyên tắc phân tách trách nhiệm, giảm sự phụ thuộc giữa các thành phần và đảm bảo khả năng mở rộng trong tương lai. Mỗi hệ thống con được xây dựng để đảm nhận một vai trò cụ thể, đồng thời giao tiếp với các hệ thống khác thông qua các giao diện rõ ràng. Cách tiếp cận này giúp hạn chế



Hình 7 : Hình mạch sử lý tác vụ

việc các thành phần phụ thuộc chặt chẽ vào nhau, từ đó giảm thiểu rủi ro phát sinh lỗi khi chỉnh sửa hoặc mở rộng hệ thống. Bên cạnh đó, kiến trúc hệ thống cũng tuân theo nguyên tắc dữ liệu và logic xử lý được tách biệt. Các dữ liệu cấu hình như thông tin kỹ năng, chỉ số nhân vật và tham số AI được lưu trữ độc lập, giúp việc cân bằng gameplay và tinh chỉnh hành vi AI trở nên thuận tiện hơn. Điều này đặc biệt quan trọng trong các dự án nghiên cứu, nơi việc thử nghiệm và điều chỉnh diễn ra thường xuyên.

2.4.2. Mô hình phân tầng của hệ thống game

Hệ thống game được tổ chức theo mô hình phân tầng, trong đó mỗi tầng đảm nhiệm một nhóm chức năng riêng biệt. Tầng trình bày chịu trách nhiệm hiển thị giao diện người dùng và phản hồi tương tác từ người chơi. Tầng logic gameplay đảm nhận việc xử lý lượt chơi, quản lý lượt và điều phối hành động của các đơn vị trong trận chiến. Tầng trí tuệ nhân tạo hoạt động độc lập, chỉ tập trung vào việc phân tích trạng thái và đưa ra quyết định hành động cho các đơn vị do máy điều khiển.

Việc phân tầng rõ ràng giúp hệ thống dễ hiểu và dễ bảo trì. Khi cần thay đổi giao diện hoặc nâng cấp mô hình AI, các tầng khác của hệ thống hầu như không bị ảnh hưởng. Điều này góp phần đảm bảo tính ổn định của game trong quá trình phát triển và thử nghiệm.

2.4.3. Thiết kế các mô-đun chức năng chính

Các mô-đun chức năng chính trong hệ thống bao gồm mô-đun quản lý scene, mô-đun quản lý trận chiến, mô-đun điều khiển nhân vật và mô-đun trí tuệ nhân tạo. Mỗi mô-đun được thiết kế để hoạt động độc lập, đồng thời cung cấp các giao diện cần thiết để tương tác với các mô-đun khác.

Mô-đun quản lý scene chịu trách nhiệm điều phối luồng gameplay, bao gồm việc chuyển đổi giữa scene cốt truyện và scene chiến đấu. Mô-đun quản lý trận chiến xử lý toàn bộ logic liên quan đến lượt chơi, hành động và kết quả chiến đấu. Mô-đun điều khiển nhân vật đảm nhận việc quản lý trạng thái và hành vi của từng đơn vị trong trận chiến. Cuối cùng, mô-đun trí tuệ nhân tạo thực hiện việc phân tích tình huống và ra quyết định hành động cho các đơn vị AI.

Cách phân chia mô-đun này giúp hệ thống có cấu trúc rõ ràng, đồng thời tạo điều kiện thuận lợi cho việc kiểm thử và mở rộng về sau.

2.5. Thiết kế hệ thống chiến đấu theo lượt

2.5.1. Tổng quan thiết kế hệ thống chiến đấu

Hệ thống chiến đấu theo lượt là trung tâm của gameplay và đóng vai trò quyết định đến trải nghiệm chiến thuật của người chơi. Trong đề tài này, hệ thống chiến đấu được thiết kế nhằm đảm bảo tính minh bạch, dễ hiểu và có chiều sâu chiến thuật. Mỗi trận chiến diễn

ra theo một chuỗi lượt được xác định rõ ràng, trong đó các đơn vị lần lượt thực hiện hành động của mình dựa trên thứ tự lượt đã được thiết lập.

Thiết kế hệ thống chiến đấu chú trọng đến việc tách biệt rõ ràng giữa các giai đoạn xử lý. Điều này giúp tránh các xung đột logic và đảm bảo rằng mọi hành động đều được xử lý đúng thứ tự. Đồng thời, cách tổ chức này tạo điều kiện thuận lợi cho việc mở rộng thêm các cơ chế chiến đấu mới trong tương lai.

2.5.2. Thiết kế vòng đời lượt chơi

Vòng đời của một lượt chơi được thiết kế gồm nhiều giai đoạn liên tiếp, bắt đầu từ việc xác định đơn vị đến lượt cho đến khi hoàn tất hành động và chuyển sang lượt tiếp theo. Mỗi giai đoạn đảm nhiệm một vai trò cụ thể và được xử lý theo trình tự cố định, giúp hệ thống kiểm soát chặt chẽ luồng chiến đấu.

Khi bắt đầu lượt, hệ thống xác định đơn vị được phép hành động và cập nhật trạng thái tương ứng. Trong giai đoạn lựa chọn hành động, người chơi hoặc hệ thống AI đưa ra quyết định dựa trên các hành động khả dụng. Sau khi hành động được xác nhận, hệ thống thực thi hành động và cập nhật các thay đổi về trạng thái. Cuối cùng, lượt chơi kết thúc và quyền hành động được chuyển sang đơn vị tiếp theo.

Việc thiết kế vòng đời lượt chơi rõ ràng giúp hệ thống dễ kiểm soát và giảm thiểu các lỗi phát sinh trong quá trình xử lý chiến đấu.

2.5.3. Thiết kế hệ thống hành động và kỹ năng

Hệ thống hành động và kỹ năng được thiết kế nhằm cung cấp cho người chơi và AI một tập hợp các lựa chọn chiến thuật đa dạng. Mỗi kỹ năng được định nghĩa bởi các thuộc tính như phạm vi tác động, hiệu ứng và điều kiện sử dụng. Các thuộc tính này được lưu trữ dưới dạng dữ liệu cấu hình, tách biệt khỏi logic xử lý, giúp việc chỉnh sửa và cân bằng gameplay trở nên thuận tiện hơn.

Khi một hành động hoặc kỹ năng được sử dụng, hệ thống sẽ kiểm tra các điều kiện cần thiết trước khi thực thi. Sau khi hành động được thực hiện, hệ thống cập nhật trạng thái của các đơn vị liên quan và ghi nhận các hiệu ứng phát sinh. Cách thiết kế này đảm bảo

rằng mọi hành động đều tuân theo luật chơi đã được định nghĩa và có thể dễ dàng mở rộng trong tương lai.

2.6. Thiết kế hệ thống trí tuệ nhân tạo theo mô hình Utility AI

2.6.1. Lý do lựa chọn mô hình Utility AI

Trong các game chiến thuật theo lượt, trí tuệ nhân tạo không chỉ cần thực hiện các hành động hợp lệ mà còn phải thể hiện được tính chiến thuật và khả năng thích ứng với tình huống. Các mô hình AI truyền thống như AI theo luật cố định hoặc FSM thường gặp hạn chế khi số lượng trạng thái và hành vi tăng lên, dẫn đến hệ thống khó mở rộng và khó bảo trì.

Mô hình Utility AI được lựa chọn trong đề tài này do khả năng đánh giá linh hoạt các hành động dựa trên giá trị utility, phản ánh mức độ “hữu ích” của từng hành động trong bối cảnh cụ thể. Thay vì ràng buộc AI vào các trạng thái cố định, Utility AI cho phép mỗi hành động được xem xét độc lập và so sánh với các hành động khác, từ đó lựa chọn phương án tối ưu nhất tại thời điểm ra quyết định.

So với các mô hình FSM hoặc Rule-Based AI, Utility AI không yêu cầu xây dựng số lượng lớn trạng thái hoặc luật điều kiện. Khi cần bổ sung kỹ năng hoặc hành vi mới, hệ thống chỉ cần thêm các hàm đánh giá tương ứng mà không phải thay đổi toàn bộ kiến trúc AI. Đây là lý do mô hình Utility AI được lựa chọn làm nền tảng cho hệ thống trí tuệ nhân tạo trong đề tài này [2][6][9].

2.6.2. Cấu trúc tổng thể của hệ thống Utility AI

Hệ thống Utility AI được thiết kế như một mô-đun độc lập, hoạt động song song với hệ thống chiến đấu nhưng không phụ thuộc trực tiếp vào logic gameplay. Tại mỗi lượt của một đơn vị AI, hệ thống sẽ thu thập các thông tin trạng thái cần thiết từ trận chiến, bao gồm vị trí các đơn vị, lượng máu, trạng thái hiệu ứng và các kỹ năng có thể sử dụng.

Dựa trên các thông tin này, hệ thống Utility AI tiến hành đánh giá từng hành động khả dụng. Mỗi hành động được gán một tập các hàm utility, phản ánh các yếu tố chiến thuật như mức độ nguy hiểm, khả năng gây sát thương hoặc khả năng hỗ trợ đồng đội. Giá trị

utility cuối cùng của một hành động được tính toán thông qua việc tổng hợp các hàm đánh giá này.

Sau khi hoàn tất quá trình đánh giá, hệ thống AI lựa chọn hành động có giá trị utility cao nhất và trả về kết quả cho hệ thống chiến đấu để thực thi. Việc tách biệt rõ ràng giữa quá trình đánh giá và thực thi giúp hệ thống AI dễ kiểm soát và dễ kiểm thử.

2.6.3. Thiết kế cơ chế lựa chọn mục tiêu

Trước khi lựa chọn hành động, hệ thống AI cần xác định mục tiêu phù hợp để tối đa hóa hiệu quả chiến thuật của lượt chơi. Thay vì lựa chọn ngẫu nhiên hoặc sử dụng các luật cố định, hệ thống sử dụng cơ chế Utility-Based Target Selection nhằm đánh giá tất cả các mục tiêu khả dụng và lựa chọn mục tiêu có giá trị chiến thuật cao nhất.

Khi đến lượt của một đơn vị AI, hệ thống sẽ thu thập danh sách tất cả các mục tiêu còn sống và tiến hành tính toán điểm số cho từng mục tiêu. Điểm số này được xây dựng từ nhiều yếu tố khác nhau nhằm phản ánh chính xác mức độ ưu tiên của mục tiêu trong bối cảnh hiện tại của trận đấu.

Các yếu tố đánh giá bao gồm:

HP Score: đánh giá dựa trên lượng máu hiện tại của mục tiêu. Những mục tiêu có lượng máu thấp sẽ nhận điểm ưu tiên cao hơn nhằm tăng khả năng kết liễu.

Threat Score: đánh giá mức độ nguy hiểm của mục tiêu đối với đội hình AI. Những nhân vật gây nhiều sát thương hoặc có ảnh hưởng lớn đến trận đấu sẽ được ưu tiên xử lý.

Finisher Bonus: điểm thưởng bổ sung khi mục tiêu đang ở trạng thái máu thấp và có khả năng bị loại khỏi trận đấu trong lượt hiện tại.

Focus Penalty: hệ số giảm điểm nhằm tránh tình trạng nhiều đơn vị AI cùng tập trung vào một mục tiêu duy nhất.

Random Noise: một giá trị ngẫu nhiên nhỏ được bổ sung nhằm giảm tính lặp lại trong hành vi của AI.

Điểm số cuối cùng của mục tiêu được xác định theo công thức:

$$\text{TargetScore} = [(\text{HPScore} \times \text{HPBias})(\text{ThreatScore} \times \text{ThreatBias})(\text{FinisherBonus} \times \text{FinisherWeight})\text{Noise}] \times \text{FocusPenalty}$$

Trong đó HPBias, ThreatBias và FinisherWeight được lấy từ Utility Profile của từng loại AI.

Sau khi hoàn tất quá trình đánh giá, hệ thống lựa chọn mục tiêu có Target Score cao nhất để sử dụng trong bước đánh giá kỹ năng tiếp theo.

Đối với một số loại AI đặc biệt như Nemesis, hệ thống sử dụng cơ chế truy sát chuyên biệt. Trong trường hợp này AI sẽ bỏ qua phần lớn các tiêu chí đánh giá thông thường và tập trung trực tiếp vào nhân vật có giá trị Threat cao nhất. Cơ chế này được sử dụng nhằm tạo cảm giác các Boss hoặc kẻ địch đặc biệt đang chủ động truy đuổi những nhân vật nguy hiểm nhất của người chơi.

Nhờ cơ chế đánh giá đa tiêu chí, hệ thống có khả năng lựa chọn mục tiêu một cách linh hoạt và phù hợp với diễn biến thực tế của trận đấu thay vì sử dụng các quy tắc cứng nhắc như các mô hình AI truyền thống.

2.6.4. Thiết kế hành vi AI theo vai trò

Sau khi xác định được mục tiêu phù hợp, hệ thống tiếp tục thực hiện quá trình đánh giá các kỹ năng khả dụng để lựa chọn hành động tối ưu cho lượt hiện tại. Mỗi kỹ năng được xem như một phương án hành động độc lập. Hệ thống lần lượt đánh giá tất cả các kỹ năng mà đơn vị AI có thể sử dụng tại thời điểm hiện tại, bao gồm kỹ năng tấn công, hồi máu, phòng thủ, gây hiệu ứng bất lợi hoặc kỹ năng diện rộng.

Để đánh giá hiệu quả của từng kỹ năng, hệ thống sử dụng tập hợp các hàm Utility Score chuyên biệt:

Attack Score: đánh giá hiệu quả gây sát thương lên mục tiêu.

Heal Score: đánh giá mức độ cần thiết của việc hồi phục cho đồng minh.

Defend Score: đánh giá lợi ích của hành động phòng thủ.

Debuff Score: đánh giá hiệu quả của các hiệu ứng bất lợi lên đối thủ.

AoE Score: đánh giá hiệu quả khi tác động lên nhiều mục tiêu cùng lúc.

Finisher Score: đánh giá khả năng kết liễu mục tiêu.

Giá trị Utility cuối cùng của một kỹ năng được tính toán theo công thức:

$$\text{SkillUtility} = (\text{AttackScore} \times \text{AttackWeight}) + (\text{HealScore} \times \text{HealWeight}) + (\text{DefendScore} \times \text{DefendWeight}) + (\text{DebuffScore} \times \text{DebuffWeight}) + (\text{AoEScore} \times \text{AoEWeight}) + (\text{FinisherScore} \times \text{FinisherWeight})$$

Trong đó các trọng số được lấy từ Utility Profile của đơn vị AI.

Khác với các mô hình AI truyền thống phải xây dựng nhiều thuật toán riêng cho từng loại nhân vật, hệ thống trong đề tài sử dụng một lõi Utility AI thống nhất kết hợp với Utility Profile nhằm tạo ra nhiều phong cách chiến đấu khác nhau.

Mỗi Utility Profile được định nghĩa thông qua tập hợp các trọng số chiến thuật riêng biệt. Việc thay đổi các trọng số này sẽ làm thay đổi cách AI đánh giá hành động mà không cần sửa đổi thuật toán cốt lõi.

Ví dụ:

Aggressive: tăng Attack Weight và Finisher Weight nhằm ưu tiên gây sát thương và kết liễu mục tiêu.

Debuffer: tăng Debuff Weight để ưu tiên sử dụng kỹ năng gây hiệu ứng bất lợi.

Assassin: ưu tiên rất cao cho Finisher Weight nhằm tập trung loại bỏ các mục tiêu máu thấp.

Shielder: tăng Defend Weight để ưu tiên các hành động phòng thủ và bảo vệ đồng minh.

Healer: tăng Heal Weight nhằm tập trung hồi máu và hỗ trợ đội hình.

Nemesis: tăng mạnh Threat Bias để truy đuổi mục tiêu nguy hiểm nhất trên chiến trường.

Sau khi hoàn tất quá trình đánh giá, kỹ năng có Skill Utility cao nhất sẽ được lựa chọn và gửi về Battle Manager để thực thi.

Cách tiếp cận này cho phép toàn bộ các đơn vị AI trong game sử dụng chung một hệ thống ra quyết định nhưng vẫn thể hiện được những phong cách chiến đấu khác nhau. Điều này giúp giảm đáng kể độ phức tạp của hệ thống, đồng thời tăng khả năng mở rộng và cân bằng gameplay trong quá trình phát triển.

2.6.5. Tích hợp Utility AI với hệ thống chiến đấu

Việc tích hợp Utility AI với hệ thống chiến đấu được thiết kế theo hướng tối giản và rõ ràng. Hệ thống chiến đấu chỉ đóng vai trò cung cấp dữ liệu trạng thái và nhận lại quyết

định hành động từ mô-đun AI. Toàn bộ quá trình đánh giá và lựa chọn hành động được thực hiện bên trong mô-đun AI, đảm bảo sự độc lập giữa hai hệ thống.

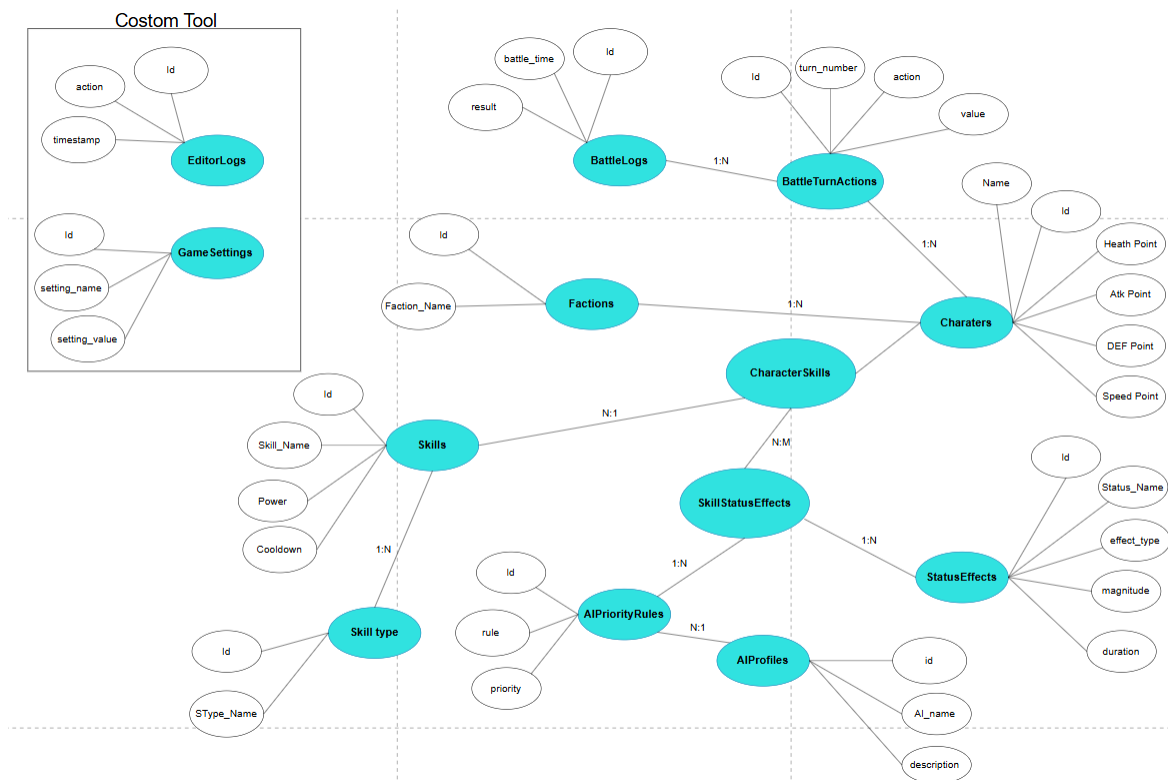
Cách tích hợp này giúp giảm sự phụ thuộc giữa AI và gameplay, đồng thời tạo điều kiện thuận lợi cho việc thay thế hoặc nâng cấp mô hình AI trong tương lai. Điều này đặc biệt quan trọng đối với các dự án nghiên cứu, nơi yêu cầu thử nghiệm và đánh giá nhiều phương pháp khác nhau.

2.7. Thiết kế dữ liệu và lớp đối tượng

2.7.1. Nguyên tắc thiết kế dữ liệu

Thiết kế dữ liệu trong hệ thống game hướng đến mục tiêu tách biệt dữ liệu cấu hình và logic xử lý. Các dữ liệu như chỉ số nhân vật, thông tin kỹ năng và tham số AI được lưu trữ độc lập, giúp việc chỉnh sửa và cân bằng gameplay trở nên linh hoạt hơn. Cách tiếp cận này cũng giúp giảm thiểu rủi ro phát sinh lỗi khi thay đổi dữ liệu.

Bên cạnh đó, dữ liệu được thiết kế để có thể tái sử dụng và mở rộng. Việc sử dụng các cấu trúc dữ liệu phù hợp giúp hệ thống dễ dàng bổ sung thêm nhân vật, kỹ năng hoặc hành vi AI mà không cần thay đổi lớn trong kiến trúc tổng thể.

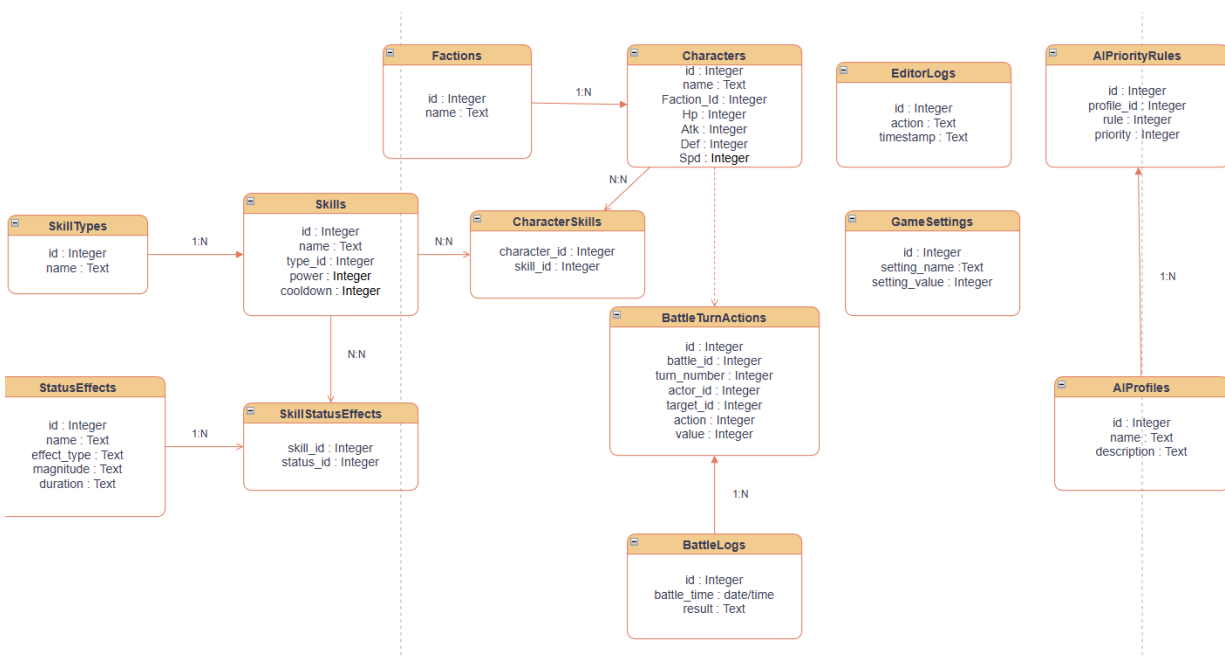


Hình 8 : Sơ đồ ERD

3.7.2. Thiết kế các lớp đối tượng chính

Các lớp đối tượng chính trong hệ thống bao gồm lớp đại diện cho đơn vị chiến đấu, lớp quản lý trận chiến và lớp xử lý AI. Mỗi lớp đảm nhiệm một vai trò rõ ràng và chỉ tập trung vào các chức năng liên quan. Cách tổ chức này giúp mã nguồn dễ hiểu và dễ bảo trì.

Lớp đại diện cho đơn vị chiến đấu chịu trách nhiệm quản lý trạng thái và hành vi của từng nhân vật. Lớp quản lý trận chiến điều phối luồng chiến đấu và xử lý lượt chơi. Lớp xử lý AI thực hiện việc đánh giá tình huống và ra quyết định hành động cho các đơn vị do máy điều khiển.



Hình 9 :Class Diagram

2.7.3. Ứng dụng Scriptable Object trong thiết kế dữ liệu

Scriptable Object được sử dụng rộng rãi trong hệ thống để lưu trữ các dữ liệu cấu hình. Việc này giúp tách biệt dữ liệu khỏi logic xử lý và cho phép chỉnh sửa trực quan thông qua Unity Editor. Trong đề tài, Scriptable Object được ứng dụng để quản lý thông tin kỹ năng, chỉ số nhân vật và tham số hành vi AI.

Cách sử dụng Scriptable Object giúp quá trình thiết kế và cân bằng gameplay trở nên hiệu quả hơn, đồng thời hỗ trợ việc mở rộng nội dung trong tương lai. Đây cũng là một trong những yếu tố giúp hệ thống đạt được tính linh hoạt và khả năng mở rộng cao.

2.8. Tổng kết chương

Chương 3 đã trình bày quá trình phân tích và thiết kế hệ thống cho đề tài phát triển game nhập vai chiến thuật theo lượt sử dụng mô hình trí tuệ nhân tạo Utility AI. Nội dung chương tập trung làm rõ các yêu cầu chức năng của hệ thống, phân tích luồng hoạt động của gameplay, đồng thời xây dựng kiến trúc tổng thể cho các thành phần cốt lõi của trò chơi.

Bên cạnh việc thiết kế hệ thống chiến đấu theo lượt, chương này cũng đã mô tả chi tiết kiến trúc và cơ chế hoạt động của hệ thống trí tuệ nhân tạo. Mô hình Utility AI được phân tích từ cấp độ tổng thể đến các thành phần cụ thể như cơ chế lựa chọn mục tiêu, cơ chế lựa chọn kỹ năng, Threat System và Utility Profile. Các thành phần này phối hợp với nhau nhằm tạo ra những hành vi chiến đấu linh hoạt, đa dạng và phù hợp với từng vai trò chiến thuật của nhân vật AI trong trận đấu.

Ngoài ra, chương cũng đã trình bày nguyên tắc phân tách giữa hệ thống chiến đấu và hệ thống AI, giúp tăng khả năng mở rộng, bảo trì và nâng cấp hệ thống trong tương lai. Việc áp dụng kiến trúc hướng mô-đun cho phép các thành phần hoạt động độc lập nhưng vẫn đảm bảo tính đồng bộ trong toàn bộ gameplay.

Những kết quả phân tích và thiết kế trong chương này là cơ sở quan trọng cho quá trình hiện thực hóa hệ thống trong Unity. Trên nền tảng đó, Chương 4 sẽ trình bày chi tiết quá trình cài đặt, xây dựng cơ sở dữ liệu, phát triển các công cụ hỗ trợ (Custom Tool) và triển khai hệ thống chiến đấu cùng mô hình Utility AI trong môi trường thực tế.

CHƯƠNG 3: XÂY DỰNG VÀ CÀI ĐẶT HỆ THỐNG

3.1 Kiến trúc tổng thể của hệ thống :

Hệ thống game được xây dựng theo hướng **component-based**, trong đó mỗi chức năng chính được tách thành các module độc lập nhưng có khả năng liên kết với nhau. Các module chính bao gồm:

Module quản lý Battle (BattleManager)

Module nhân vật và chỉ số (CharacterStats, BattleUnit)

Module AI và ra quyết định (BattleAI, UtilityPresets)

Module giao diện người dùng (BattleUI, UnitUI)

Module dữ liệu (SQLite Database)

cách tiếp cận này giúp hệ thống dễ mở rộng, dễ bảo trì và thuận lợi cho việc tinh chỉnh gameplay.

3.2 Xây dựng hệ thống Battle :

3.2.1 BattleManager – trung tâm điều phối trận đấu :

Trong hệ thống hiện tại, lớp BattleManager đóng vai trò trung tâm, chịu trách nhiệm quản lý toàn bộ vòng đời của một trận chiến. Ngay khi Battle Scene được load, BattleManager khởi tạo kết nối tới cơ sở dữ liệu SQLite đặt trong thư mục *StreamingAssets*, từ đó truy xuất dữ liệu nhân vật, kỹ năng và AI profile.

Quá trình khởi tạo trận đấu được thực hiện theo trình tự: tải dữ liệu nhân vật từ database, spawn các đơn vị lên Battle Scene thông qua prefab, sau đó khởi tạo thứ tự lượt và bắt đầu trận chiến bằng coroutine *BattleIntro*. Việc sử dụng coroutine cho phép hệ thống tạo ra khoảng trễ hợp lý trước khi trận đấu chính thức diễn ra, giúp tăng tính mạch lạc cho trải nghiệm người chơi.

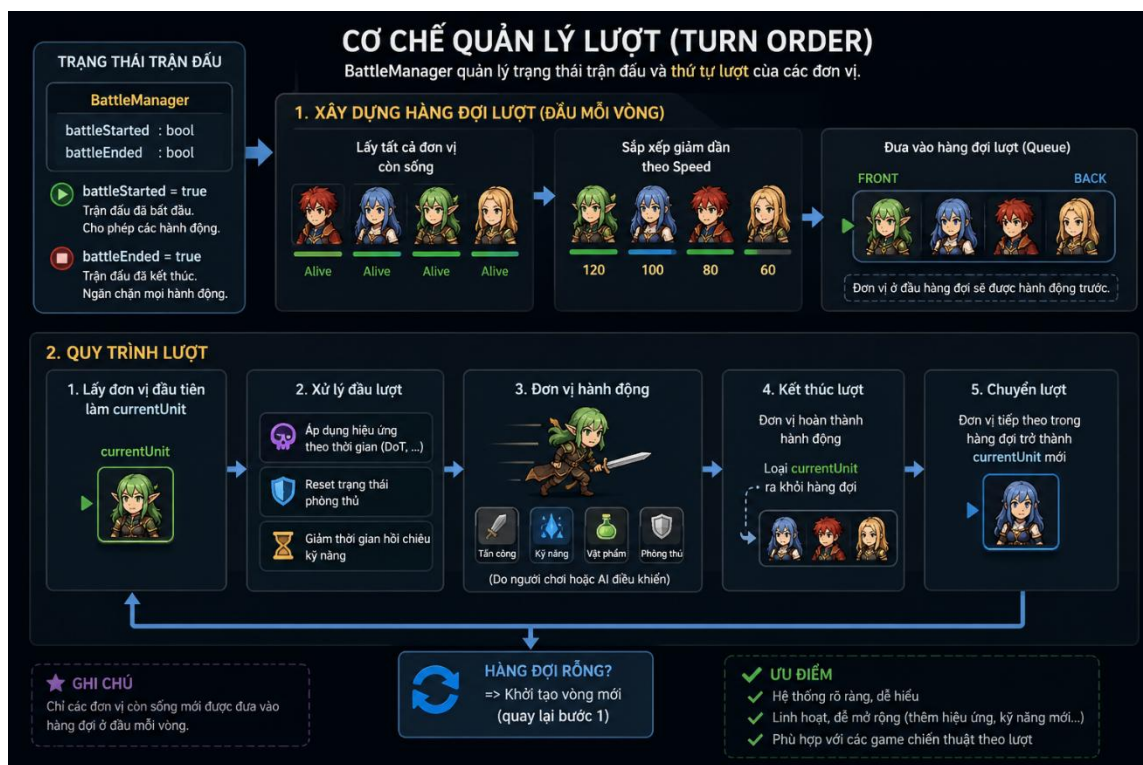
3.2.2 Cơ chế quản lý lượt (Turn Order) :

BattleManager cũng chịu trách nhiệm theo dõi trạng thái trận đấu thông qua các biến *battleStarted* và *battleEnded*, từ đó ngăn chặn các hành động không hợp lệ khi trận đấu chưa sẵn sàng hoặc đã kết thúc.

Hệ thống turn-based được hiện thực hóa bằng một hàng đợi (*Queue*) chứa các BattleUnit. Thứ tự lượt được xác định dựa trên chỉ số *Speed* của từng nhân vật. Tại đầu mỗi vòng, BattleManager gom toàn bộ đơn vị còn sống, sắp xếp giảm dần theo tốc độ và đưa vào hàng đợi lượt.

Trong mỗi lượt, BattleManager lấy đơn vị đầu tiên trong hàng đợi làm *currentUnit*. Trước khi đơn vị hành động, hệ thống xử lý các hiệu ứng theo thời gian như DoT, đồng thời reset trạng thái phòng thủ và giảm thời gian hồi chiêu của kỹ năng. Sau khi đơn vị hoàn thành hành động, quyền điều khiển được chuyển sang đơn vị tiếp theo. Khi hàng đợi rỗng, một vòng mới sẽ được khởi tạo.

Để mà nói thì cơ chế của nó chỉ đơn giản là thuật toán FCFS nhưng được thêm thắt và chỉnh sửa để tạo thành một cơ chế game



Hình 10 :Cơ chế hàng đợi trong turn based

Phân tích hình ảnh trên thì bắt đầu trận thì cũng như thuật toán FCFS thì nó tạo một hàng đợi sau đó cứ liên tục thêm biến vào hàng đợi theo thứ tự thông qua hệ thống so sánh chỉ số tốc độ giữa các nhân vật , khi tới lượt thì thực hiện toàn bộ những hành động được lập trình , người chơi sẽ chọn hành động ,tùy thêm nữa là nếu có hiệu ứng đầu turn thì cũng sẽ kích hoạt trước khi vào lượt hoặc trước khi kết thúc lượt để chuyển sang nhân vật khác .

3.3. Xây dựng hệ thống nhân vật :

3.3.1 CharacterStats

```
public CharacterStats(int id, int faction_id, string name, int hp, int atk, int def, int spd, int ai_profile_id)
{
    this.id = id;
    this.faction_id = faction_id;
    this.name = name;
    maxHP = hp;
    currentHP = hp;
    attack = atk;
    defense = def;
    speed = spd;
    this.ai_profile_id = ai_profile_id;
}
```

Các chỉ số trên là thiết lập để lấy dữ liệu từ database sau đó tham chiếu để đem vào trò chơi , nó rất quan trọng vì trò chơi cần những chỉ số này để thực sự hoạt động .

```
public void TakeDamage(int damage)
{
    if (isDefending)
    {
        damage = Mathf.RoundToInt(damage * defenseMultiplier);
    }

    currentHP -= damage;
    if (currentHP < 0) currentHP = 0;
}

0 references
public void Heal(int amount)
{
    currentHP += amount;
    if (currentHP > maxHP) currentHP = maxHP;
}
```

Hàm này nhằm tính toán thiệt hại , người chơi và kẻ thù sau khi bị tấn công sẽ gọi hàm này để hệ thống kiểm tra và chỉnh sửa lại hpbar sau đó cập lại giá trị để tiếp tục hoặc kết thúc dựa trên điều kiện đặt ra

3.3.2 BattleUnit :

BattleUnit là lớp đại diện cho nhân vật trong Battle Scene, kết nối giữa dữ liệu (CharacterStats), hiển thị (Sprite/Animation) và tương tác gameplay. Mỗi BattleUnit có thể:

Nhận lệnh di chuyển và tấn công
Hiển thị hiệu ứng sát thương, hồi máu
Được chọn làm mục tiêu bởi người chơi hoặc AI
Cập nhật các trạng thái của Hpbar
Hiển thị sát thương gây ra .

```
public void Highlight(bool active)
{
    StopAllCoroutines();

    if (active)
    {
        StartCoroutine(BlinkEffect());
    }
    else
    {
        sr.color = Color.white; // reset
    }
}

1 reference
IEnumerator BlinkEffect()
{
    while (true)
    {
        sr.color = Color.yellow;
        yield return new WaitForSeconds(0.2f);
        sr.color = Color.white;
        yield return new WaitForSeconds(0.2f);
    }
}

public void TakeDamage(int amount, DamagePopup.PopupType type = DamagePopup.PopupType.Normal)
{
    stats.TakeDamage(amount);
    if (stats.currentHP < 0) stats.currentHP = 0;

    PlayAnim("Hurt");

    Vector3 popupPos = transform.position + new Vector3(0, 2f, 0);

    PopupManager.Instance.ShowDamage(
        popupPos,
        amount,
        type
    );
    Debug.Log($"[TakeDamage] {stats.name} amount={amount} type={type}");

    UpdateHPBar();
}
```

3.4 Xây dựng hệ thống kỹ năng :

Hệ thống kỹ năng trong game được thiết kế theo hướng data-driven, trong đó toàn bộ thông tin kỹ năng được lưu trữ trong cơ sở dữ liệu SQLite. Mỗi kỹ năng được mô tả bởi các thuộc tính như loại kỹ năng (Attack, Heal, Buff, Defense, DoT, Cure), kiểu mục tiêu (Single, AOE, Self), phạm vi tấn công (Melee hoặc Ranged), sức mạnh và thời gian hồi chiêu.

Khi đến lượt người chơi, BattleManager cung cấp danh sách kỹ năng của nhân vật hiện tại cho BattleUI. Khi người chơi kích hoạt một kỹ năng, hệ thống sẽ kiểm tra tính hợp lệ của hành động, bao gồm điều kiện mục tiêu và trạng thái hồi chiêu. Nếu kỹ năng hợp lệ, BattleManager tiến hành xử lý logic tương ứng theo từng loại kỹ năng.

Các kỹ năng tấn công có thể yêu cầu di chuyển nhân vật tới gần mục tiêu (đối với melee) thông qua coroutine *MoveAndAttack*, từ đó tạo cảm giác hành động trực quan hơn. Sau khi gây sát thương, hệ thống cập nhật bảng Threat và giao diện người dùng.

Đối với các kỹ năng hỗ trợ như Heal, Buff hay Defense, hiệu ứng được áp dụng trực tiếp lên chỉ số nhân vật. Các kỹ năng gây hiệu ứng theo thời gian (DoT) được lưu trong danh sách hiệu ứng của nhân vật và được xử lý tự động ở đầu mỗi lượt.

```
public void UseSkill(SkillData uiSkill)
{
    if (currentUnit == null || !currentUnit.isPlayer)
    {
        Debug.LogWarning($"[UseSkill] Bỏ qua: currentUnit null hoặc không phải người chơi. uiSkillId={uiSkill != null ? uiSkill.id : -1}");
        return;
    }

    var skill = currentUnit.skills.FirstOrDefault(s => s.id == uiSkill.id);
    if (skill == null)
    {
        Debug.LogError($"[UseSkill] Không tìm thấy skill id={uiSkill?.id} trong danh sách skill của {currentUnit.stats.name}");
        return;
    }

    var attacker = currentUnit.stats;
    var targetUnit = selectedTarget;
    var caster = currentUnit;

    // Kiểm tra cooldown
    if (skill.currentCooldown > 0)
    {
        ui.ShowMessage(skill.name + " chưa hồi, còn " + skill.currentCooldown + " lượt!");
        Debug.Log($"[UseSkill] Skill {skill.name} bị chặn do cooldown còn {skill.currentCooldown}");
        return;
    }

    Debug.Log($"[UseSkill] Sub-nhánh: Melee > MoveAndAttack");
    StartCoroutine(MoveAndAttack(currentUnit, targetUnit, () =>
    {
        int damage = Mathf.RoundToInt(attacker.attack * (skill.power / 100f));
        var target = targetUnit.stats;
        targetUnit.TakeDamage(damage);
        BattleManager.Instance.AddThreat(attacker, damage);
        enemyUIItems[enemyTeam.IndexOf(target)].UpdateHP();

        ui.ShowMessage(attacker.name + " gây " + damage + " sát thương bằng " + skill.name + "!");
        Debug.Log($"[UseSkill] Damage (melee single): {damage} lên {target.name}");
        NextTurn();
    }));
}
```

```

else // Ranged
{
    Debug.Log("[UseSkill] Sub-nhánh: Ranged → không di chuyển");
    int damage = Mathf.RoundToInt(attacker.attack * (skill.power / 100f));
    var target = targetUnit.stats;
    targetUnit.TakeDamage(damage);
    BattleManager.Instance.AddThreat(attacker, damage);
    enemyUIItems[enemyTeam.IndexOf(target)].UpdateHP();

    ui.ShowMessage(attacker.name + " gây " + damage + " sát thương tầm xa bằng " + skill.name + "!");
    Debug.Log($"[UseSkill] Damage (ranged single): {damage} lên {target.name}");
    NextTurn();
}

e if (skill.targetType == TargetType.AOE)

Debug.Log("[UseSkill] Nhánh: Attack + AOE");
int hits = 0;
int totalDamage = 0;
foreach (var enemy in enemyUnits)
{
    if (enemy.stats.IsDead()) continue;
    int damage = Mathf.RoundToInt(attacker.attack * (skill.power / 100f));
    enemy.TakeDamage(damage);
    totalDamage += damage;
    enemyUIItems[enemyTeam.IndexOf(enemy.stats)].UpdateHP();
    hits++;
    Debug.Log($"[UseSkill] AOE hit: {enemy.stats.name} nhận {damage}");
}
BattleManager.Instance.AddThreat(attacker, totalDamage);

ui.ShowMessage(attacker.name + $" tung {skill.name}, trúng {hits} mục tiêu!");
NextTurn();

ak;

void ApplyDOT(SkillData skill, CharacterStats attacker, BattleUnit target)
{
    if (target == null) return;

    int dotDamage = Mathf.RoundToInt(attacker.attack * (skill.power / 100f));

    target.stats.AddDOT(
        damagePerTurn: dotDamage,
        turns: 3,
        source: attacker );

    ui.ShowMessage($"{attacker.name} gây Độc lên {target.stats.name}!");
}

void ApplyCure(SkillData skill, CharacterStats attacker, BattleUnit target)
{
    if (target == null) return;
    target.stats.dots.Clear();
}

if (!TryValidateTarget(caster, targetUnit, skill))
    return;

```

```

case SkillType.Heal:
    if (skill.targetType == TargetType.Single)
    {
        Debug.Log("[UseSkill] Nhánh: Heal + Single");
        if (targetUnit == null || !targetUnit.isPlayer)
        {
            ui.ShowMessage("Hãy chọn đồng đội để hồi máu!");
            Debug.LogWarning("[UseSkill] Heal Single nhưng target null hoặc không phải đồng minh");
            return;
        }

        var healTarget = targetUnit.stats;
        int before = healTarget.currentHP;

        healTarget.currentHP = Mathf.Min(
            healTarget.currentHP + skill.power,
            healTarget.maxHP
        );

        int actualHeal = healTarget.currentHP - before;

        targetUnit.ShowHeal(actualHeal);

        playerUIItems[playerTeam.IndexOf(healTarget)].UpdateHP();

        ui.ShowMessage(attacker.name + " hồi phục " + skill.power + " HP cho " + healTarget.name + "!");
        ui.UpdateTeamHP(playerTeam, enemyTeam);
        Debug.Log($"[UseSkill] Heal Single: {healTarget.name} {before} -> {healTarget.currentHP}");

        targetUnit.Highlight(false);
        selectedTarget = null;
        NextTurn();
    }
else if (skill.targetType == TargetType.AOE)
{
    Debug.Log("[UseSkill] Nhánh: Heal + AOE");
    int healed = 0;
    foreach (var ally in playerUnits)
    {
        if (ally.stats.IsDead()) continue;

        int before = ally.stats.currentHP;

        ally.stats.currentHP = Mathf.Min(
            ally.stats.currentHP + skill.power,
            ally.stats.maxHP
        );

        int actualHeal = ally.stats.currentHP - before;

        ally.ShowHeal(actualHeal);

        playerUIItems[playerTeam.IndexOf(ally.stats)].UpdateHP();
        healed++;
        Debug.Log($"[UseSkill] Heal AOE: {ally.stats.name} {before} -> {ally.stats.currentHP}");
    }

    ui.UpdateTeamHP(playerTeam, enemyTeam);
    ui.ShowMessage(attacker.name + $" tung {skill.name}, hồi cho {healed} đồng minh!");
    NextTurn();
}

```

3.5. Xây dựng hệ thống AI Enemy

3.5.1. AI dựa trên Utility

AI của enemy được xây dựng dựa trên mô hình **Utility-based AI**, trong đó mỗi hành động được đánh giá thông qua một hàm utility thay vì các nhánh điều kiện cứng. Mỗi enemy được gán một *AI Profile* thông qua trường *ai_profile_id*, cho phép định nghĩa hành vi chiến đấu khác nhau cho từng loại kẻ địch.

Trong mỗi lượt của enemy, BattleManager gọi đến các hàm trong lớp BattleAI để lựa chọn kỹ năng và mục tiêu. Việc lựa chọn mục tiêu không mang tính ngẫu nhiên mà dựa trên nhiều yếu tố như lượng Threat mà player tạo ra và mức độ tập trung tấn công (Focus).

```
public static float ScoreAttack(BattleUnit caster, BattleUnit target, Dictionary<int, int> threat)
{
    if (target == null || target.stats.IsDead()) return 0f;
    float hpFactor = 1f - target.stats.HPPercent(); // máu càng th?p càng ?u tiên
    float threatFactor = threat.ContainsKey(target.stats.id) ? threat[target.stats.id] / 100f : 0f;
    return hpFactor * 0.7f + threatFactor * 0.3f;
}

0 references
public static float ScoreAOE(List<BattleUnit> targets)
{
    int alive = targets.Count(u => !u.stats.IsDead());
    float avgHpLoss = targets.Where(u => !u.stats.IsDead())
        .Average(u => 1f - u.stats.HPPercent());
    return (alive / 5f) + avgHpLoss * 0.5f;
}

0 references
public static float ScoreHeal(List<BattleUnit> allies)
{
    var lowest = allies.Where(a => !a.stats.IsDead())
        .OrderBy(a => a.stats.HPPercent())
        .FirstOrDefault();
    return lowest != null ? 1f - lowest.stats.HPPercent() : 0f;
}

0 references
public static float ScoreDefend(BattleUnit caster)
{
    return caster.stats.HPPercent() < 0.35f ? 1f : 0.2f;
}

0 references
public static float ScoreDebuff(BattleUnit target)
{
    if (target == null) return 0f;
    return target.stats.defense / 100f;
}
```

Đoạn code trên đang thể hiện các biến trong một chuỗi so sánh giữa các hành động khả dĩ cho AI, tùy preset mà các trọng số sẽ khác nhau mà dẫn đến kết quả so sánh khác nhau dẫn đến hành động cũng khác nhau.


```

foreach (var p in alive)
{
    var stats = p.stats;

    // 1) HP SCORE Low Hp detectk
    float hpScore = (1f - stats.HPPercent()) * 0.50f;

    // 2) THREAT SCORE Ain't low check threat
    float threatRaw = threatTable.ContainsKey(stats.id)
        ? (float)threatTable[stats.id] / stats.maxHP
        : 0f;

    float threatScore = Mathf.Clamp(threatRaw, 0f, 0.40f);

    // 3) Too much hit ! Stop
    int fc = focusCount.ContainsKey(stats.id) ? focusCount[stats.id] : 0;
    float focusPenalty =
        (fc >= 3) ? 0.15f :
        (fc == 2) ? 0.40f :
        (fc == 1) ? 0.75f : 1f;

    // 4) FINISHER BONUS - too low HP must focus
    float finisher = (stats.HPPercent() < 0.25f) ? 0.20f : 0f;

    // 5) random
    float noise = Random.Range(0.0f, 0.15f);

    // 6) FINAL SCORE - yes
    float finalScore =
    (
        hpScore * util.hpBias +
        threatScore * util.threatBias +
        finisher * util.finisherWeight +
        noise
    ) * focusPenalty;
    Debug.Log(
        $"[AI DEBUG] Target: {p.stats.name} | " +
        $"HP%: {p.stats.HPPercent():0.00} | " +
        $"HPScore: {hpScore:0.00} | " +
        $"ThreatRaw: {threatRaw:0.00} | ThreatScore: {threatScore:0.00} | " +
        $"Finisher: {finisher:0.00} | Noise: {noise:0.00} | " +
        $"Focus: {fc} | Penalty: {focusPenalty:0.00} | " +
        $"FINAL: {finalScore:0.000}"
    );
    if (finalScore > bestScore)
    {
        bestScore = finalScore;
        best = p;
    }
}
if (best != null)
{

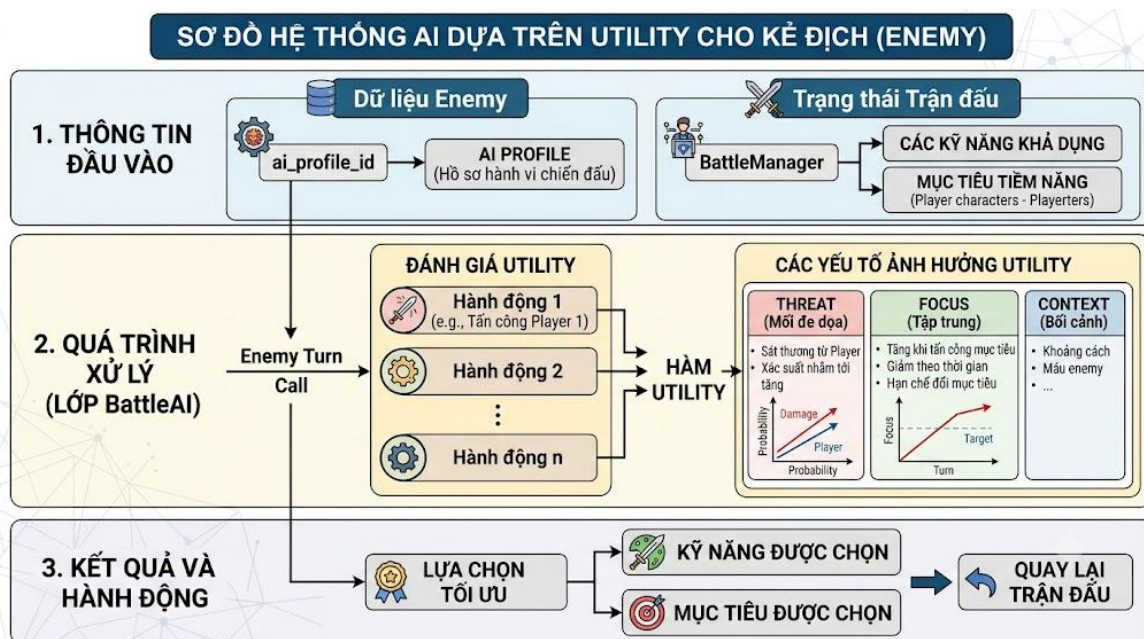
```

Nếu mà đã có preset và đã kiểm tra xong tất cả các thông tin được cung cấp thì sau so sánh các điểm số hành động và bắt đầu thực thi hành động trên mục tiêu mà nó cũng đã dùng utility để chọn ra mục tiêu mà AI thực hiện hành động lên , ví dụ như healer với preset healscore rất cao nên khi bắt đầu nhận được thông tin là có một đồng đội dưới năm mươi phần trăm thì sẽ kiểm tra bản thân có skill dạng heal đang không cooldown không , nếu có thì heal cho mục tiêu đó , nếu không có skill khả dụng thì trường hợp cho biến đó sẽ được đổi và thực hiện cơ chế chống crash .

3.5.2. Threat và Focus

Hệ thống Threat ghi nhận tổng sát thương mà mỗi nhân vật player gây ra lên enemy. Khi enemy cần chọn mục tiêu, các nhân vật có Threat cao sẽ có xác suất bị nhắm tới lớn hơn, tạo cảm giác AI biết "tra đũa" những mục tiêu nguy hiểm.

Bên cạnh đó, chỉ số Focus được sử dụng để hạn chế việc enemy đổi mục tiêu liên tục giữa các lượt. Focus tăng lên mỗi khi một mục tiêu bị tấn công và giảm dần theo thời gian, giúp AI duy trì sự nhất quán trong hành vi.



Hình 11 : sơ đồ luồng sử lý của AI

3.5.3. Minh họa về quá trình ra quyết định của hệ thống Utility AI

Để minh họa cách hệ thống Utility AI hoạt động trong trò chơi, giả sử tại một thời điểm trong trận chiến, trùm cuối đang đối đầu với ba nhân vật của người chơi gồm Knight, Archer và Healer. Trạng thái hiện tại của các nhân vật được thể hiện như sau:

Knight còn 90% HP với chỉ số Threat bằng 80 và đã bị hai đồng minh tập trung tấn công trước đó (Focus Count = 2);

Archer còn 40% HP, Threat bằng 30 và chưa bị tập trung (Focus Count = 0);

Healer chỉ còn 20% HP, Threat bằng 60 và đã bị tấn công một lần trước đó (Focus Count = 1).

Hồ sơ Utility của Boss được thiết lập với các trọng số gồm hpBias = 1.0, threatBias = 0.8 và finisherWeight = 1.2.

Khi bắt đầu lượt của trùm, hệ thống lần lượt duyệt qua từng mục tiêu còn sống và tính Utility Score theo công thức đã được cài đặt trong hàm SelectUtilityTarget(). Đầu tiên, đối với Knight, lượng máu còn lại khá cao nên điểm ưu tiên theo HP được tính là:

$$HPScore = (1 - 0.9) \times 0.5 = 0.05$$

Chỉ số Threat sau khi chuẩn hóa là:

$$ThreatScore = \frac{80}{100} \times 0.4 = 0.32$$

Do Knight còn trên 25% HP nên không nhận được điểm thưởng kết liễu (Finisher Bonus = 0). Ngoài ra, vì Knight đã bị hai đồng minh tập trung tấn công nên hệ số Focus Penalty chỉ còn 0,4. Giả sử giá trị ngẫu nhiên (Noise) sinh ra là 0,06 thì Utility Score cuối cùng được tính như sau:

$$Utility = ((0.05 \times 1.0) + (0.32 \times 0.8) + (0 \times 1.2) + 0.06) \times 0.4 \approx 0.15$$

Đối với Archer, lượng máu chỉ còn 40% nên:

$$HPScore = (1 - 0.4) \times 0.5 = 0.30$$

Threat Score được tính:

$$ThreatScore = \frac{30}{100} \times 0.4 = 0.12$$

Archer chưa đủ điều kiện kích hoạt Finisher Bonus và chưa bị tập trung tấn công nên hệ số Focus Penalty bằng 0,75. Giả sử Noise bằng 0,04 thì Utility Score thu được là:

$$Utility = ((0.30 \times 1.0) + (0.12 \times 0.8) + 0 + 0.04) \times 0.75 \approx 0.33$$

Cuối cùng, đối với Healer, do chỉ còn 20% HP nên:

$$HPScore = (1 - 0.2) \times 0.5 = 0.40$$

Threat Score được tính là:

$$ThreatScore = \frac{60}{100} \times 0.4 = 0.24$$

Vì HP nhỏ hơn 25%, Healer nhận thêm Finisher Bonus bằng 0,20. Với hệ số Focus Penalty bằng 0,75 và Noise bằng 0,05, Utility Score được tính như sau:

$$Utility = ((0.40 \times 1.0) + (0.24 \times 0.8) + (0.20 \times 1.2) + 0.05) \times 0.75 \approx 0.66$$

Dựa trên các phép tính trên, có thể thấy mục tiêu có số điểm cao nhất là Healer với 0.66

3.6 Hình ảnh Gameplay của dự án :



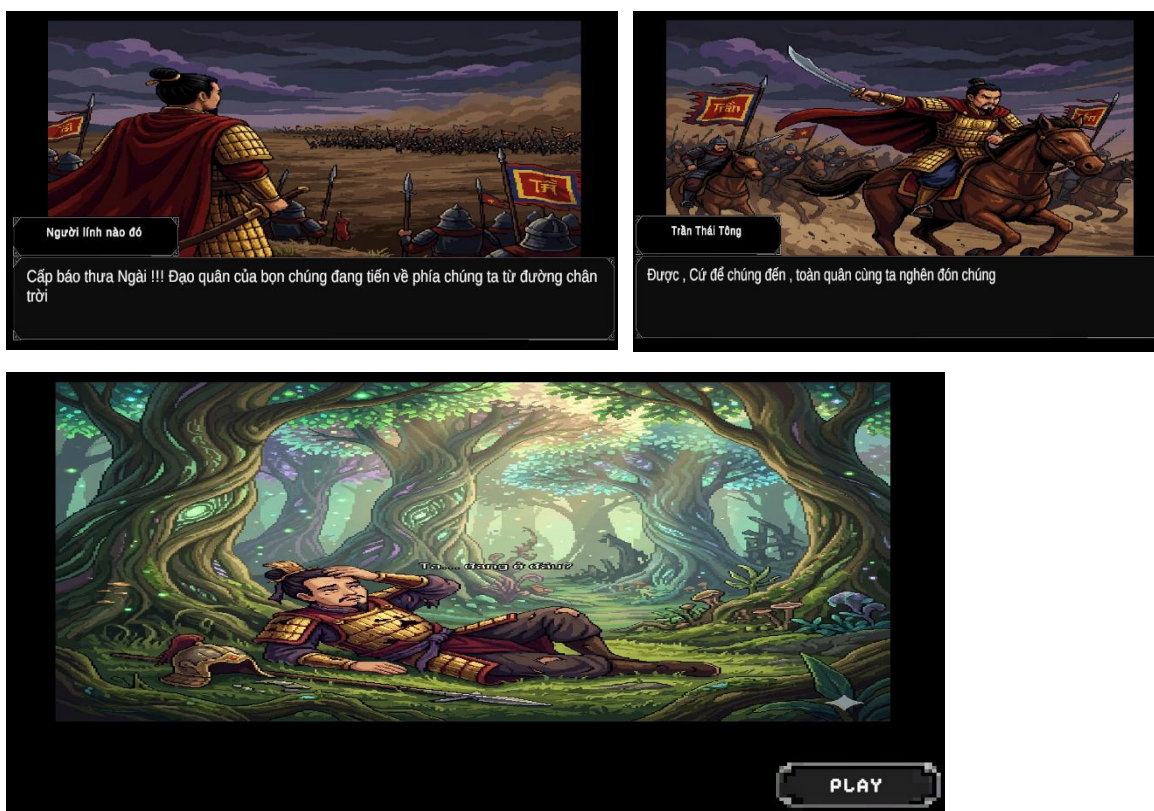
Hình 12: Main Menu

Sau khi chọn new game thì sẽ được chuyển sang Starter Panel để chọn nhân vật khởi đầu để bắt đầu trò chơi, nếu chọn continue thì sẽ mở trang save/load để chọn file đã load .



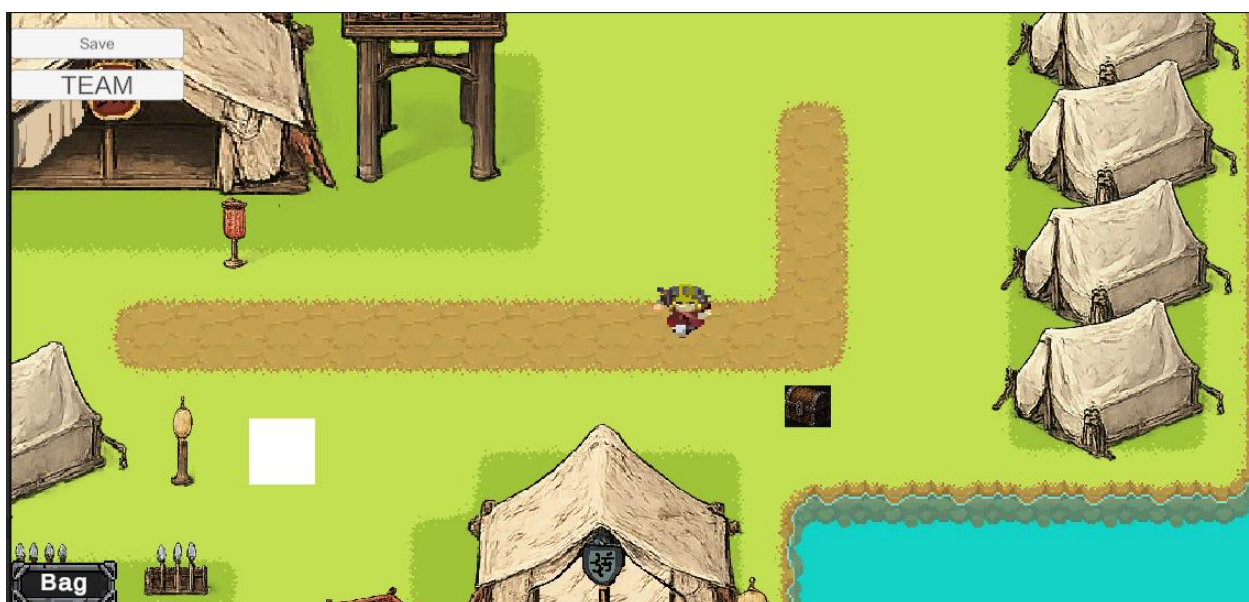
Hình 13 Starter Panel

Có thể chọn một giữa ba nhân vật khởi đầu và panel này show ra các kỹ năng nhân vật này có cùng với các chỉ số cơ bản để dễ so sánh , khi chọn xong và bấm start thì sẽ chạy intro .



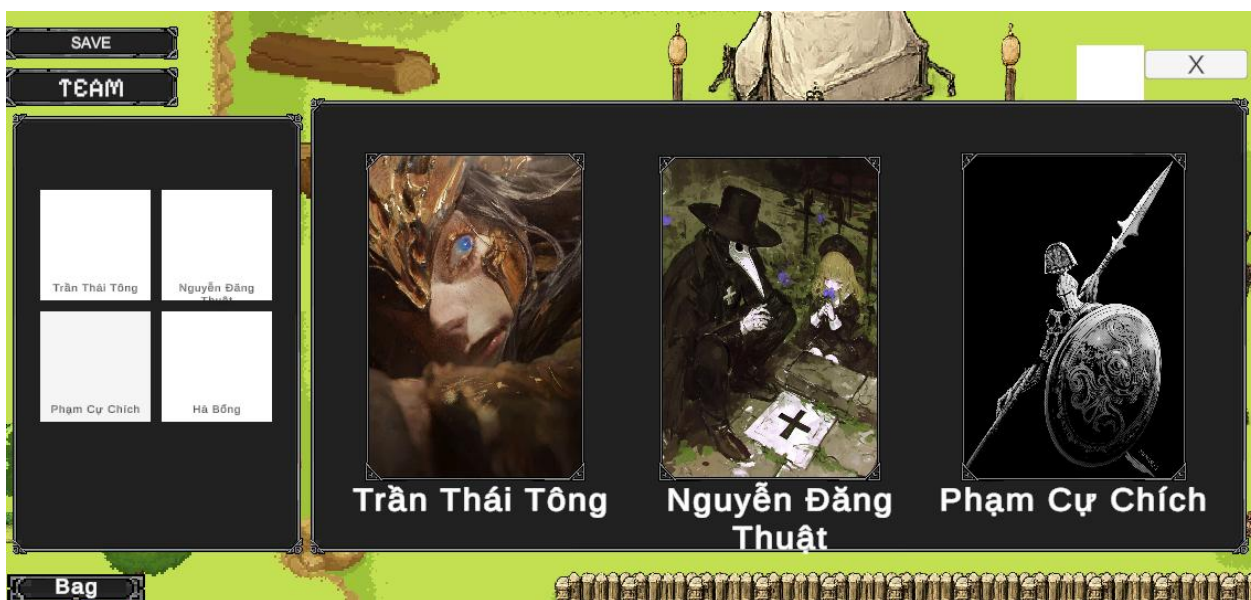
Hình 14 Intro Panel cốt truyện

Sau khi đã chọn thì các đoạn intro cho từng nhân vật được chạy dựa trên nhân vật đã chọn và sau khi bấm play thì sẽ chuyển sang story scene.



Hình 15 Main scene - Story Scene

Đây là scene tương tác chính của người chơi , tương tác với NPC hoặc trigger các trận đánh và bật các inventory ,sắp xếp team



Hình 16 :Team building panel

Ở panel này thì cho phép người chơi sắp xếp đội hình với các nhân vật đã mở khóa , sau đó có thể chọn một nhân vật trong team để mở character panel để có thể trang bị vật phẩm đang sở hữu .



Hình 17 Character Panel

Đây là panel để người chơi trang bị các vật phẩm sưu tầm được thông qua quá trình chơi .

Bên trái là stat panel để hiện chỉ số

Panel mid là các vật phẩm có trong inventory

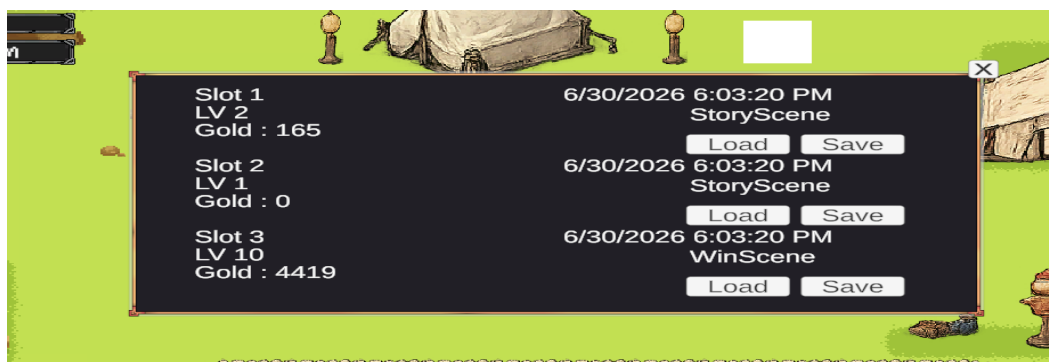
Panel phải là hình ảnh của nhân vật

Dưới panel nhân vật là panel để chuyển đổi nhân vật



Hình 18 Quest panel

Giúp người chơi kiểm tra và biết được việc cần làm sau khi nhận được nhiệm vụ từ NPC



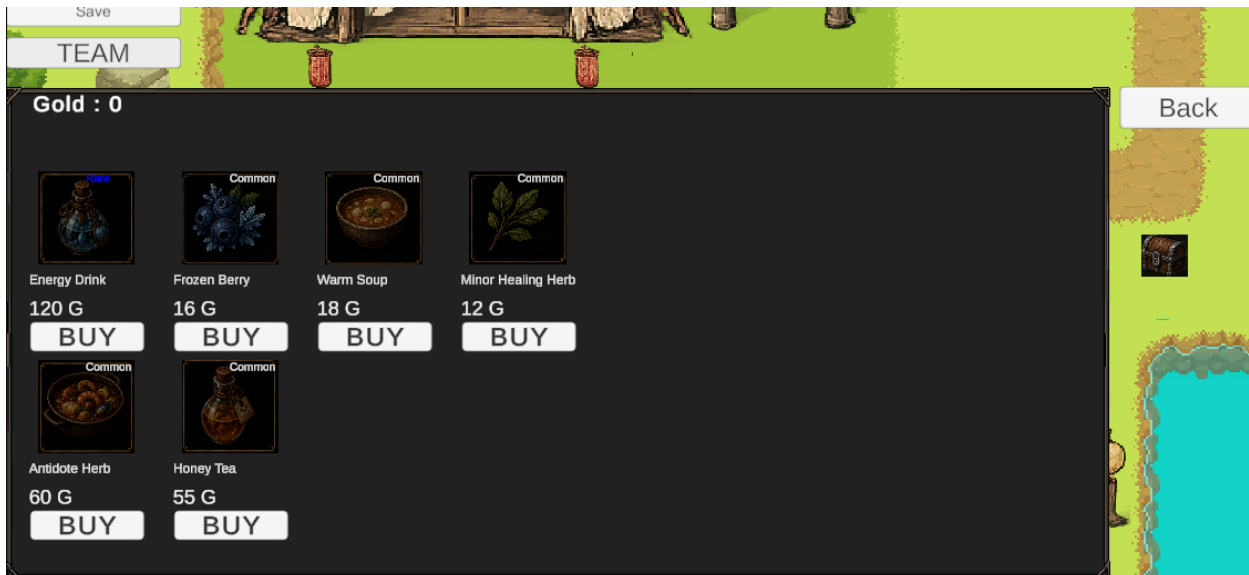
Hình 19 Save/Load panel

Save Panel để người chơi lưu hoặc load các file đã lưu trước đó, file đó lưu toàn bộ mọi thứ của người chơi của người chơi .



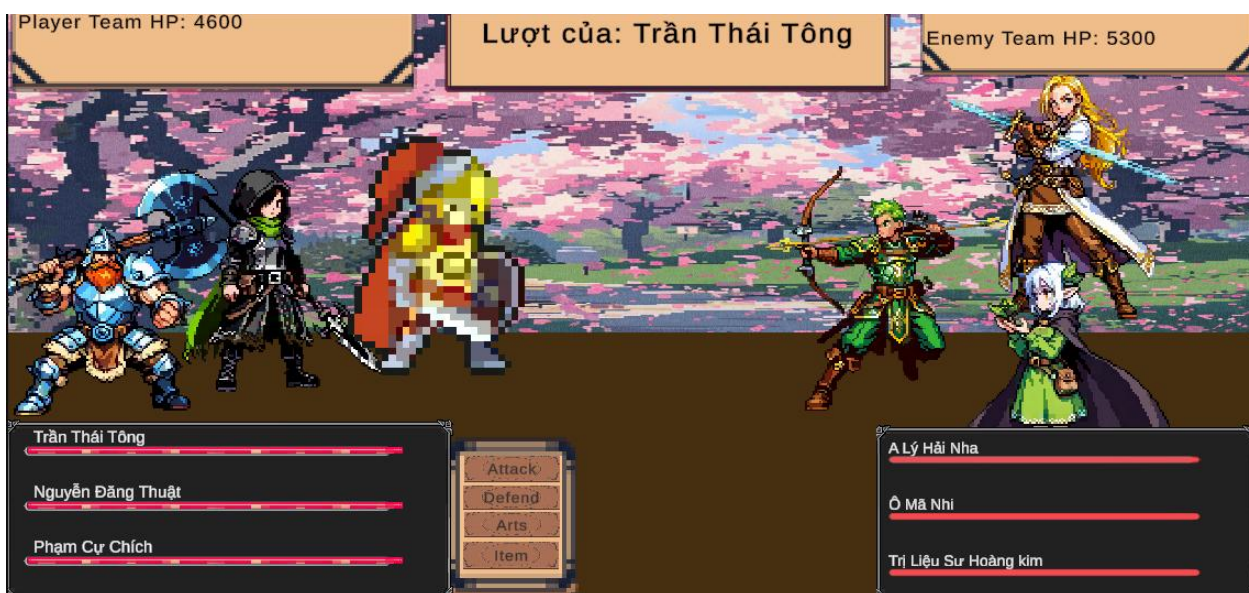
Hình 20 : Inventory

Các vật phẩm người chơi có được qua mọi hành động trong lúc chơi sẽ được lưu lại trong inventory panel



Hình 21 Shop Panel

Shop panel xuất hiện khi người chơi chọn nói chuyện với các NPC dạng bán hàng , với các loại mặt hàng khác nhau dựa trên NPC như nếu đó là NPC có tag “ weaponsmith” thì của hàng sẽ hiện các vật phẩm dạng trang bị tấn công , các NPC cũng có phân chia cấp bậc để nếu đó là một cấp bậc cao thì shop sẽ xuất hiện những vật phẩm có cấp độ cao hơn .



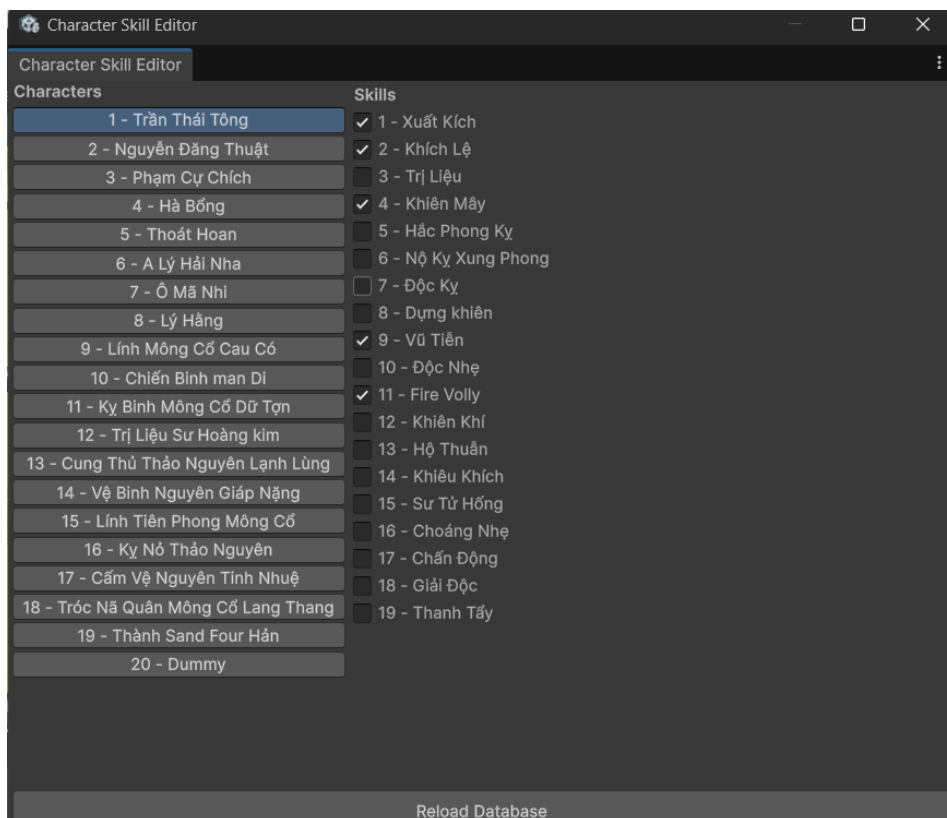
Hình 22 Main Battle Scene

Nơi người chơi thực hiện các hành động với các enemy là kẻ cản đường người chơi



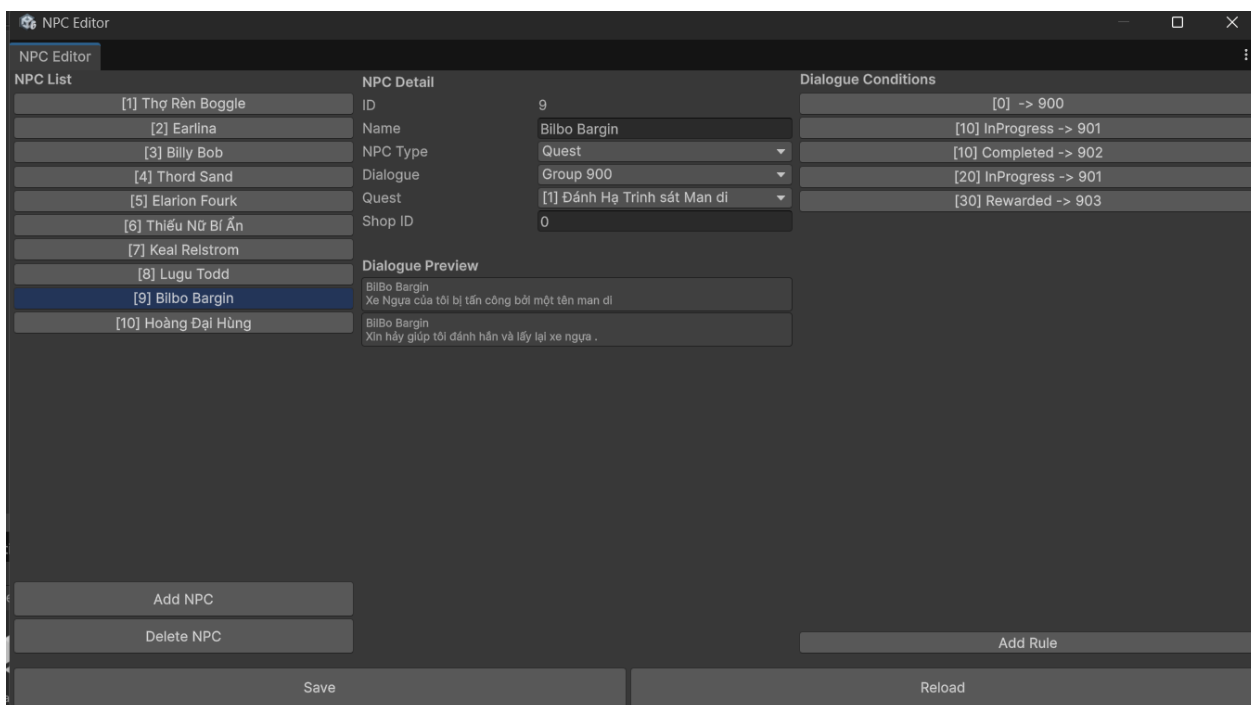
Hình 23 Skill Panel

Người chơi chọn Arts để mở skill panel trong đó hiện các skill mà nhân vật đó sở hữu cùng với tool tip để người chơi đọc thông tin skill .



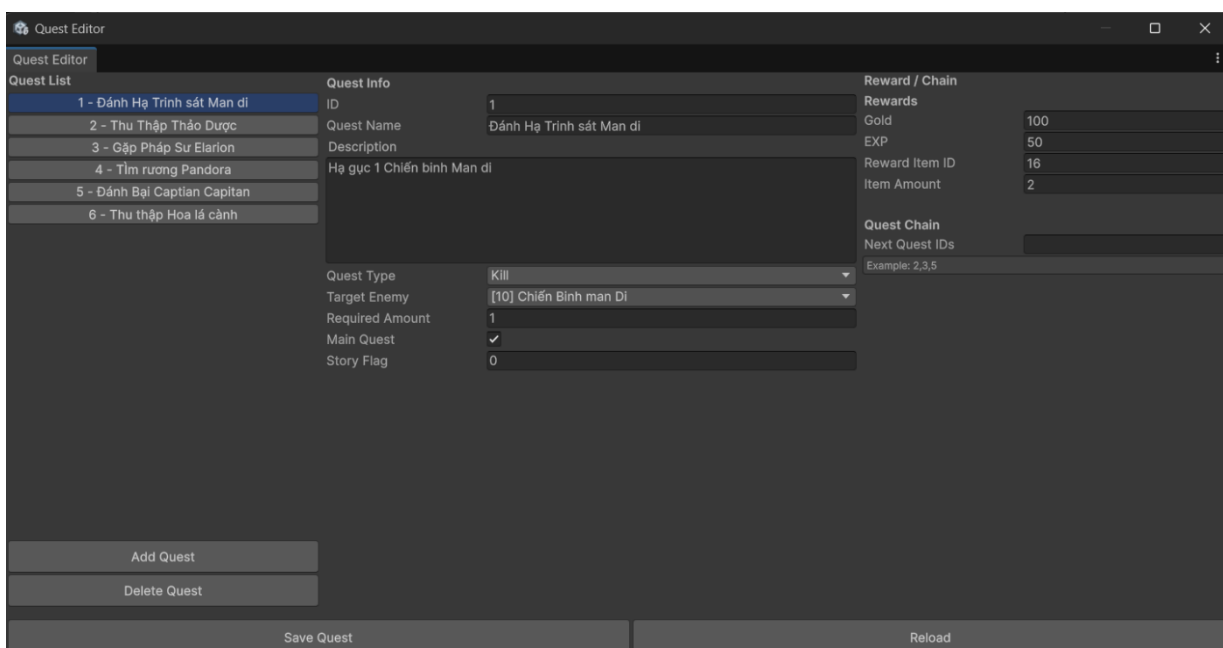
Hình 24 Skill Editor

Dùng cho việc sửa ,thay đổi skill sở hữu của một nhân vật nào đó thông qua các check box



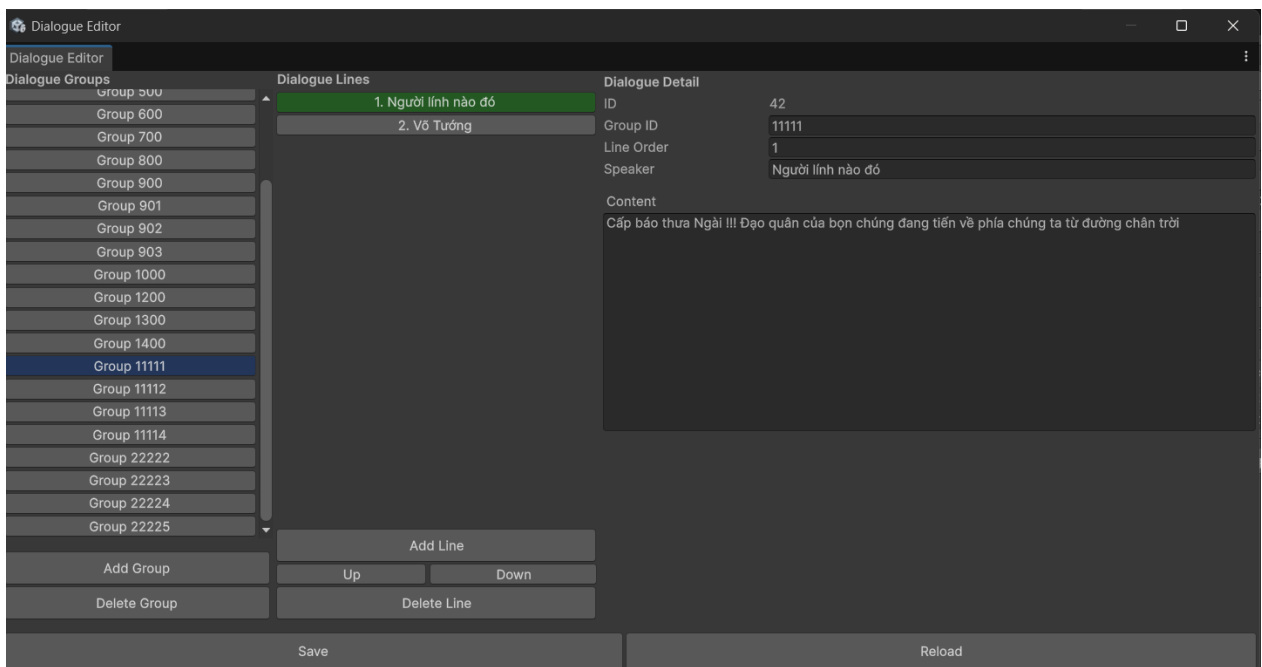
Hình 25 NPC Editor

Dùng cho việc thêm sửa xóa các NPC và chỉnh sửa các hội thoại của họ , nếu đó là npc giao quest thì sẽ là chọn quest để giao .



Hình 26 Quest Editor

Dùng cho công việc thêm sửa xóa các loại quest và phần thưởng để có thể gán cho npc để giao cho người chơi .



Hình 27 Dialog Editor

Dùng để thêm sửa xóa các đoạn hội thoại, thay đổi thứ tự hội thoại của NPC hoặc là dùng cho các đoạn cutscene

CHƯƠNG 4: KẾT LUẬN VÀ KHUYẾN NGHỊ

4.1. Kết quả đạt được

Sau quá trình nghiên cứu, phân tích, thiết kế và triển khai, đề tài đã xây dựng được một prototype game chiến thuật theo lượt hoàn chỉnh ở mức cơ bản. Hệ thống game đáp ứng đầy đủ các yêu cầu chức năng đã đề ra, bao gồm gameplay chiến đấu theo lượt, cơ chế chuyển đổi giữa story scene và battle scene, cùng với hệ thống trí tuệ nhân tạo cho kẻ địch dựa trên mô hình Utility AI.

Về mặt gameplay, hệ thống chiến đấu theo lượt được triển khai ổn định với luồng xử lý rõ ràng. Người chơi có thể điều khiển nhân vật thực hiện các hành động như tấn công, sử dụng kỹ năng và phòng thủ theo đúng thứ tự lượt. Việc quản lý lượt dựa trên chỉ số tốc độ giúp tạo ra sự khác biệt chiến thuật giữa các nhân vật, đồng thời làm tăng chiều sâu trong quá trình ra quyết định của người chơi.

Hệ thống kỹ năng được xây dựng theo hướng dữ liệu hóa, cho phép dễ dàng mở rộng và điều chỉnh thông số mà không cần thay đổi nhiều logic xử lý. Các loại kỹ năng khác nhau như tấn công, hồi phục, buff, phòng thủ và gây hiệu ứng theo thời gian đã được triển khai và hoạt động đúng với thiết kế ban đầu.

Đối với hệ thống trí tuệ nhân tạo, việc áp dụng mô hình Utility AI đã mang lại hành vi enemy linh hoạt và có tính chiến thuật hơn so với các AI dựa trên luật cứng. Enemy có khả năng lựa chọn mục tiêu và hành động dựa trên trạng thái trận đấu, chỉ số Threat và mức độ Focus, từ đó tạo ra cảm giác đối thủ có khả năng đánh giá và phản ứng trước hành động của người chơi.

4.2. Đánh giá mức độ hoàn thành mục tiêu

Dựa trên các mục tiêu nghiên cứu đã đề ra ở Chương 1, có thể nhận thấy đề tài đã đạt được phần lớn các mục tiêu ban đầu. Trước hết, đề tài đã nghiên cứu và áp dụng thành công cơ chế Turn-Based Strategy trong một game hoàn chỉnh, từ khâu thiết kế đến triển khai thực tế. Các khái niệm về vòng đời lượt, quản lý hành động và xử lý trạng thái đã được hiện thực hóa rõ ràng trong hệ thống.

Bên cạnh đó, mô hình Utility AI đã được nghiên cứu và ứng dụng hiệu quả vào hệ thống enemy AI. Việc thiết kế AI theo hướng đánh giá utility giúp hệ thống linh hoạt, dễ mở rộng và tránh được sự lặp lại hành vi thường gặp ở các mô hình AI truyền thống. Mục tiêu xây dựng kiến trúc game rõ ràng, dễ mở rộng cũng đã được đáp ứng thông qua việc phân tách các module và áp dụng mô hình component-based của Unity.

Tuy nhiên, do phạm vi đề tài giới hạn trong khuôn khổ một dự án thực tập tốt nghiệp, game mới chỉ dừng lại ở mức prototype và chưa được phát triển đầy đủ về nội dung và độ hoàn thiện.

4.3. Hạn chế của đề tài

Mặc dù đạt được các kết quả nhất định, đề tài vẫn còn tồn tại một số hạn chế. Thứ nhất, nội dung gameplay còn tương đối đơn giản và số lượng màn chơi chưa nhiều. Điều này khiến trải nghiệm người chơi chưa thực sự phong phú và chưa thể khai thác hết tiềm năng của hệ thống chiến đấu và AI đã xây dựng.

Thứ hai, hệ thống AI mặc dù đã áp dụng Utility AI nhưng vẫn sử dụng các hàm đánh giá ở mức tương đối đơn giản. AI chưa có khả năng dự đoán hành động của người chơi hoặc lập kế hoạch dài hạn cho nhiều lượt liên tiếp. Ngoài ra, hệ thống chưa tích hợp các cơ chế học hỏi hoặc điều chỉnh hành vi dựa trên lịch sử trận đấu.

Cuối cùng, phần đồ họa và âm thanh của game chưa được đầu tư. Giao diện người dùng vẫn mang tính chức năng, chưa đạt mức độ hoàn thiện cao về mặt thẩm mỹ. Điều này là phù hợp với định hướng tập trung vào gameplay và AI, nhưng cũng là một điểm hạn chế nếu xét trên góc độ sản phẩm game hoàn chỉnh.

4.4. Hướng phát triển trong tương lai

Trong tương lai, đề tài có thể được mở rộng và phát triển theo nhiều hướng khác nhau. Về gameplay, hệ thống chiến đấu có thể được bổ sung thêm các cơ chế mới như địa hình ảnh hưởng đến chiến thuật, hiệu ứng môi trường hoặc các trạng thái nâng cao cho nhân vật. Việc mở rộng số lượng kỹ năng và loại nhân vật cũng sẽ giúp tăng chiều sâu chiến thuật cho game.

Đối với hệ thống trí tuệ nhân tạo, Utility AI có thể được nâng cấp bằng cách kết hợp với các mô hình khác như Behavior Tree hoặc GOAP để tạo ra hành vi phức tạp hơn. Ngoài ra, việc tinh chỉnh các hàm utility và trọng số theo từng giai đoạn của trận đấu sẽ giúp AI có khả năng thích ứng tốt hơn với chiến thuật của người chơi.

Về mặt kỹ thuật, hệ thống có thể được tối ưu hiệu năng và mở rộng sang các nền tảng khác. Việc bổ sung lưu trữ dữ liệu người chơi, hệ thống save/load hoặc mở rộng sang chế độ nhiều người chơi cũng là những hướng phát triển tiềm năng trong tương lai.

Toàn bộ chương này đã trình bày các kết quả đạt được, đánh giá mức độ hoàn thành mục tiêu, chỉ ra những hạn chế còn tồn tại và đề xuất các hướng phát triển cho đề tài trong tương lai. Những nội dung này khép lại báo cáo thực tập tốt nghiệp và cho thấy tiềm năng tiếp tục nghiên cứu và hoàn thiện hệ thống game chiến thuật theo lượt kết hợp trí tuệ nhân tạo đã được xây dựng.

PHỤ LỤC

HƯỚNG DẪN CÀI ĐẶT

1. Yêu cầu hệ thống

Phần cứng

CPU: Intel Core i3 thế hệ 8 hoặc tương đương trở lên.

RAM: Tối thiểu 8 GB.

Dung lượng ổ cứng: Tối thiểu 2 GB trống.

GPU: Hỗ trợ DirectX 11.

Phần mềm

Hệ điều hành: Windows 10 hoặc Windows 11 (64-bit).

Unity Editor: Phiên bản 2022.3 LTS (hoặc phiên bản sử dụng trong đồ án).

Visual Studio 2022.

Git (nếu tải mã nguồn từ kho lưu trữ).

2. Cài đặt môi trường

Bước 1. Cài đặt Unity Hub

- Tải và cài đặt Unity Hub.
- Đăng nhập tài khoản Unity.

Bước 2. Cài đặt Unity Editor

- Cài đặt Unity Editor đúng phiên bản của dự án.
- Chọn các module:
 - Windows Build Support (IL2CPP)
 - Microsoft Visual Studio

Bước 3. Mở dự án

- Khởi động Unity Hub.
- Chọn Open Project.
- Chọn thư mục chứa dự án.

Unity sẽ tự động import tài nguyên và biên dịch mã nguồn.

3. Cấu hình cơ sở dữ liệu

Hệ thống sử dụng SQLite để lưu trữ dữ liệu nhân vật, kỹ năng, nhiệm vụ và hội thoại.

Đảm bảo tệp cơ sở dữ liệu được đặt đúng trong thư mục:

Assets/StreamingAssets/

4. Build trò chơi

Trong Unity:

File → Build Settings

Platform: Windows

Add Open Scenes

Build

Sau khi hoàn tất sẽ thu được file thực thi (.exe).

5. Chạy trò chơi

Mở thư mục Build.

Chạy file Game.exe.

Hệ thống sẽ tự động nạp dữ liệu và hiển thị màn hình chính.

HƯỚNG DẪN SỬ DỤNG

1. Khởi động trò chơi

Sau khi chạy chương trình, người chơi sẽ được đưa đến màn hình chính với các chức năng:

New Game

Continue

Exit

2. Điều khiển nhân vật

Di chuyển

- W A S D hoặc phím mũi tên.

Tương tác

- Phím E để tương tác với NPC hoặc các đối tượng.

Mở menu

- Esc để mở menu tạm dừng.

3. Chiến đấu

Khi gặp kẻ địch, hệ thống chuyển sang Battle Scene.

Người chơi lần lượt chọn:

- Attack
- Skill
- Item
- Defend

Sau khi xác nhận hành động, lượt sẽ chuyển sang đơn vị tiếp theo.

4. Quản lý đội hình

Tại giao diện Party:

- Thêm hoặc loại bỏ nhân vật.
- Tối đa 3 nhân vật trong đội hình.
- Có thể thay đổi trước mỗi trận chiến.

5. Quản lý trang bị

Người chơi có thể:

- Trang bị vũ khí.
- Trang bị giáp.
- Trang bị phụ kiện.
- Xem thay đổi chỉ số sau khi trang bị.

6. Nhiệm vụ

Tại cửa sổ Quest:

- Xem nhiệm vụ đang thực hiện.
- Theo dõi tiến độ.
- Nhận phần thưởng sau khi hoàn thành.

7. Lưu và tiếp tục trò chơi

Dữ liệu sẽ được lưu tự động sau khi:

- Kết thúc trận chiến.
- Hoàn thành nhiệm vụ.
- Kích hoạt sự kiện cốt truyện.

Khi khởi động lại trò chơi, chọn **Continue** để tiếp tục dữ liệu đã lưu.

8. Một số lưu ý

- Không xóa các tệp dữ liệu trong thư mục của trò chơi.
- Không thay đổi cấu trúc thư mục Build sau khi xuất bản.
- Nếu trò chơi không khởi động, kiểm tra phiên bản DirectX và Visual C++ Redistributable trên máy tính.

TÀI LIỆU THAM KHẢO

- [1]. Tracy Fullerton, *Game Design Workshop*, 5th Edition, A K Peters/CRC Press, 2024
- [2]. Ian Millington, John Funge, *Artificial Intelligence for Games*, 2nd Edition, CRC Press, 2009.
- [3]. Code Monkey, Simple Turn-Based RPG Battle System, Youtube.com
- [4]. xOctoManx, Unity Turn Based Battle/combat, Youtube.com
- [5]. Jesse Schell, *The Art of Game Design*.
- [6]. David “Rez” Graham Dave Mark, Kevin Dill, *An Introduction to Utility Theory*, Game AI Pro, CRC Press, 2014.
- [7]. Unity Technologies, *Unity User Manual*,
Truy cập tại: <https://docs.unity3d.com/Manual/index.html>
- [8]. Unity Technologies, *Unity Scripting API*,
Truy cập tại: <https://docs.unity3d.com/ScriptReference/>
- [9]. Dave Mark, *Behavioral Mathematics for Game AI*, Course Technology, 2009.
- [10]. Michael Buckland, *Programming Game AI by Example*, Jones & Bartlett Learning, 2004.
- [11]. Millington, I., *AI for Turn-Based Strategy Games*, GDC Vault, 2010.
- [12]. Dave Mark, Intrinsic Algorithm Utility AI Articles
- [13]. SQLite Consortium, *SQLite Documentation*,
Truy cập tại: <https://www.sqlite.org/docs.html>